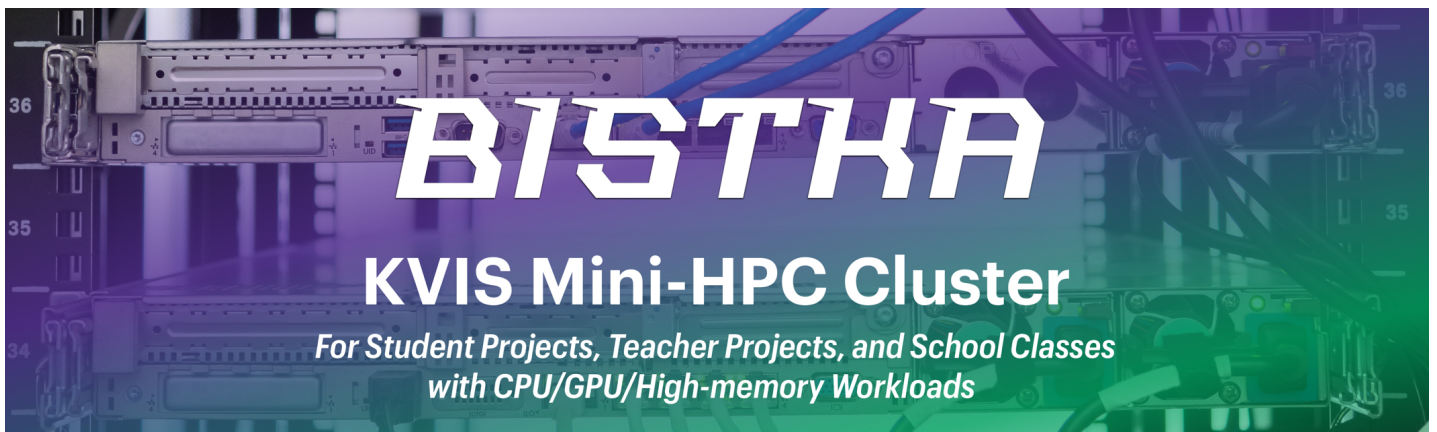


Home & Announcements

Welcome to BistKA



Important!!!

All users should read ALL Documentation on this website before starting to use BistKA.

All users must acknowledge the KVIS Mini-HPC [policies](#) and agree to comply with all rules and regulations.

The [KVIS Mini-HPC Cluster \(BistKA\)](#) is a dedicated high-performance computing environment designed to support advanced computational work at KVIS. It provides CPU, GPU, and high-memory resources for data-intensive and simulation-based workloads that exceed the capabilities of standard desktop machines. The cluster exists specifically to enhance research project and learning in computationally demanding classes, enabling students and project teams to run real-world scientific, engineering, and data science workloads.

Announcements

NOTICE

INFO

MAINTENANCE

12/11/25: The SLURM partition for [RTX PRO 4500](#) , including [gpu4500-normal](#) , [gpu4500-long](#) , and [gpu4500-devel](#) , was configured.

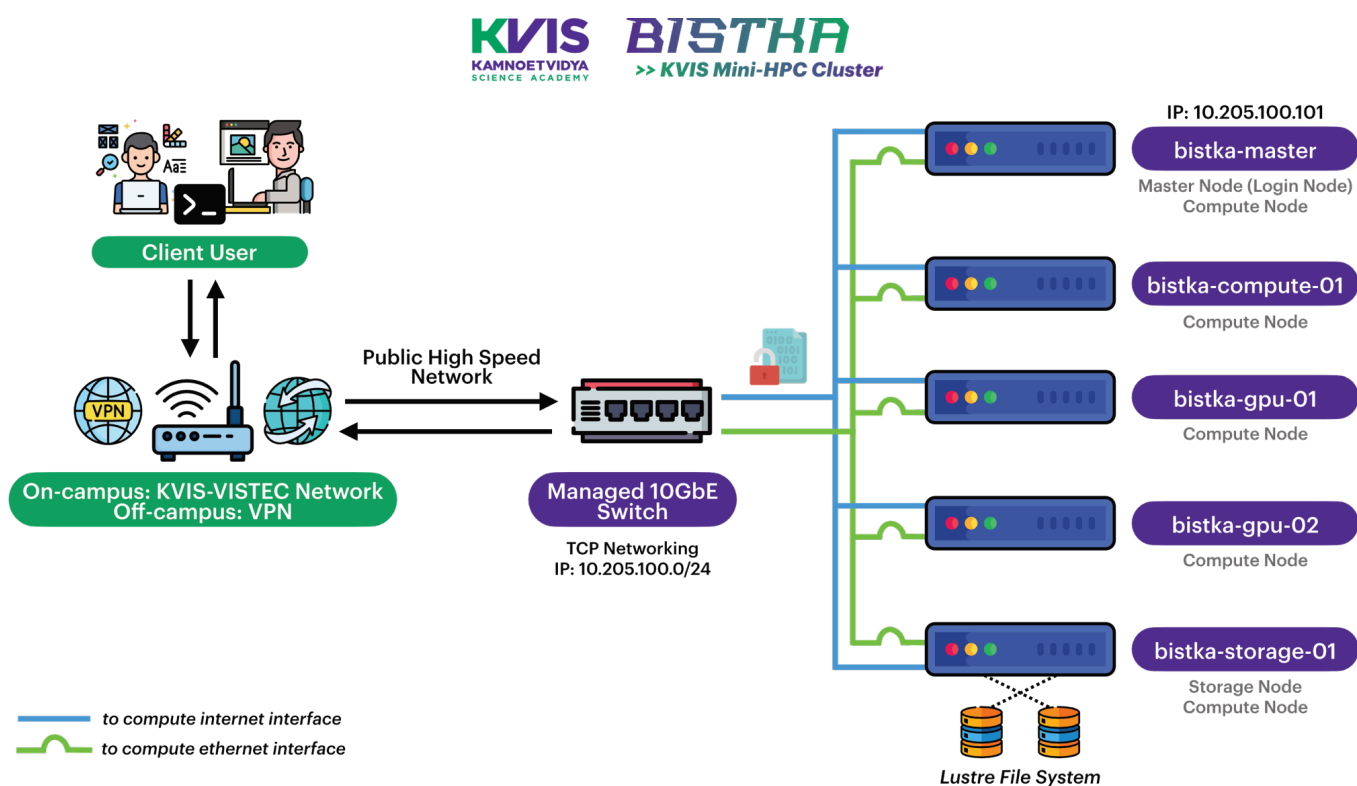
11/11/25: The 4 x [RTX PRO 4500](#) GPUs were installed to [bistka-gpu-01](#) node.

05/11/25: During this period, while the [BistKA](#) system is being tested, the cluster may not operate optimally.

Cluster Information

Cluster Information

Cluster Architecture



Cluster Specification

Hostname	Node Type	Processor	Number of socket(s)	Cores per socket(s)	Total Cores/Threads	Memo (RAM)
bistka-master (10.205.100.101)	Master node, Compute	Intel® Xeon®	2	20	40C/80T	128 GiB

	node	Gold 6138				
bistka-storage-01	Storage node, Compute node	Intel® Xeon® Gold 6138	2	20	40C/80T	128 GiB
bistka-compute-01	Compute node	AMD EPYC™ 7542	2	32	64C/128T	512 GiB
bistka-gpu-01	Compute node	AMD EPYC™ 9124	1	16	16C/32T	96 GiB
bistka-gpu-02	Compute node	AMD EPYC™ 9124	1	16	16C/32T	96 GiB

BistKA Software Stack



BistKA Software Stack							
HPC Programming Tools and Software	Software & Applications	Amber, GROMACS, Gaussian, VASP, ...					TensorFlow, PyTorch, ...
	Development & Performance Tools	Intel® oneAPI	GNU	CMake	NVIDIA® CUDA®	...	Mamba (Python, R)
	Scientific & Communication Libraries	Intel® MPI	Open MPI	MPICH	MVAPICH2	...	
Schedulers, File Systems, and Management	Environment Management	Lmod, EasyBuild, ...					
	Resource Management & Job Scheduler	SLURM					
	File Systems	Lustre®		NFS		Local (ext4, XFS)	
	Cluster Management	OpenHPC (MUNGE, Warewulf, pdsh, prun, LosF)					
Operating System and Drivers	Drivers & Network Management	Accelerator Software Stack and Driver			Lustre Networking (LNet)		
	Operating System	Linux® – Rocky Linux 9.6 (Blue Onyx)					

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

Getting Started

Getting Started

For example, to submit a Python file named `testpython.py`.

Log In to User Log-in Account

1. Open your preferred shell environment (e.g., CMD, Zsh, or Terminal). You may choose an alternative software from [Login & File Transfer](#)
2. Use `$ ssh` follow by your `username` to connect the master node.

```
1 | $ ssh <username>@10.205.100.101
```

3. Insert your password. If successful, you will see

```
1 | .
2 |      .+###-.
3 |      ## ####.
4 |      .### ####.
5 |      -#####.
6 |      -##### ##.
7 |      .##### ##.
8 |      .....+ #####.
9 |      #####-
10 |      #####.
11 |      -#####.
12 |      .#####.
13 |      .#####.
14 |      .#####-
15 |      .#####.
16 |      -#####-
17 |      .#####+
18 |      -#####.

      -----
      /  _  ) ( _ ) _ _ _ _ / / _ / // _ / /   |
      /  _  | / // _ _ // _ // , <  / / | |   |
      / / _ / // / ( _ _ ) / / _ / / | | / _ _ |
      / _ _ _ // // _ _ _ / \ _ // _ / | _ / _ / | _ |
      _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
      / // _ / | | / // _ // _ _ /
      / , <  | | / / / / \ _ \
      / / | | | / / _ / / _ _ / /
```

```
#####+ /_/_|_| |___//___/ /___/
-#####.
.#####.
#####-
.-#####.
-#####-..
.# -....-##-.-###
.## ##.
## ##
.###. .###.
..-#####-.. ..-#####-..
```

ALERT! You are entering a secured area! Your IP and login information have been recorded. System administration has been notified.

This system is restricted to authorized access only.

```
#####
#
# Welcome to BistKA Mini-HPC Cluster
# Kamnoetvidya Science Academy (KVIS)
#
# This system is running Rocky Linux 9.6 (Blue Onyx)
# Kernel is 5.14.0-570.49.1.el9_6.x86_64
# Hostname is master
#
# Quick Start and Job Submission Guide: http://10.205.100.101
# Software folder: /apps
# Modules/Packages list: $ml av
#
# BistKA Helpdesk: hpc@kvis.ac.th
# (Please see guideline via website for how to submit support service
ticket)
#
```

Announcement:

* During this period, while the system is being tested, the cluster may not operate optimally.

Policy:

- * All job execution must be submitted via job scheduler
- * Initial Credit (SHr) for Job Submission
 - User Credit: 500 SHr
 - Project Credit: 10000 SHr

```

64 * Job submission
65   Maximum Job Submission: 5 jobs/group
66   Maximum Job Running: 2 jobs/group
67   Maximum CPU-cores = 32 CPU-cores/group
68   Maximum GPUs = 2 GPUs/group
69   Maximum Walltime by Partition
70     - compute-normal: 2 days | compute-long: 4 days | compute-devel: 6
71     hours
72     - gpu4500-normal: 2 days | gpu4500-long: 4 days | gpu4500-devel: 6
73     hours
74     Default CPU-core|RAM|Walltime: 2 CPU-cores|1 GB|30 minutes
75
76 Basic Commands for Job and Storage:
77 * Submit Job: $sbatch
78 * Check Partition and its Own Policy: $sinfo
79 * Cancel Job: $scancel <jobid>
80 * Submit Job via Interactive Mode: $sinteractive
81 * Check Disk Quota: $myquota
    * Check Job Status: $myqueue
    * Check Credit: $mycredit

[hpcuser01@bistka-master ~]$

```

Your Disk Space

- Go to your home directory (this is the default directory you connect to).

```
1 | $ cd /lustrefs/home/<username>
```

- Go to your group directory.

```
1 | $ cd /lustrefs/project/<groupname>
```

Submit a Job

On the [BistKA](#), applications are not run directly on the login (master) node. Instead, users submit jobs to the cluster's SLURM workload manager, which then dispatches those jobs to the appropriate compute nodes. A job describes both the scientific or engineering work to be done (which program

to run, what input data to use, where to write output) and the computing resources required (number of CPU cores, amount of memory, GPU type and count, maximum run time, partition, etc.).

1. Edit or upload [testpython.py](#) in your disk space.
2. Prepare a script in your disk space (can use `$ vi script.sub`) to allocate resources and specify the order of execution.

You can get the script by either:

- ▶ Download the [script.sub](#) template,
- ▶ Creating a job script using the [SLURM script generator](#) , or
- ▶ Copying it from the provided example and edit via `$ vi` or `$ vim`

```

1  #!/bin/bash
2
3  # Submit this script with:
4  # sbatch <this_script.sub>
5
6  ##### SLURM header #####
7  ##### Auto generated for BistKA | KVIS Mini-HPC Cluster #####
8
9  #SBATCH --job-name=job_1          # Job name
10 #SBATCH --output=job_1_run        # Standard output file (%j = JobID)
11 #SBATCH --nodes=1                # Number of node
12 #SBATCH --cpus-per-task=2        # Number of CPU cores
13 #SBATCH --partition=compute-devel # Partition (queue)
14 #SBATCH --account=hpcuser01      # SLURM account
15 #SBATCH --mem=1GB                # Memory per job
16 #SBATCH --time=1:00:00           # Max walltime (hh:mm:ss)
17
18 #####
19
20 # Load modules
21 module load miniforge3/25.3.1-0
22
23 python testpython.py             # change file to run here

```

3. Check the list of files in the current folder via `$ ls`

```

1  $ ls
2  script.sub  testpython.py

```

4. Enter `$ sbatch` to submit your job.

```
1 | $ sbatch script.sub
```

5. If successful, use `$ myqueue` or `$ squeue` to check the status of your job.

```
1 | $ myqueue
2 |
3 |          JOBID PARTITION      NAME      USER ST      TIME  NODES
4 | NODELIST(REASON)
      80 compute-d    job_1 hpcuser01  R      0:07      1
      bistka-master
```

Log Out the BistKA System

Always log out after use by `$ exit`.

```
1 | $ exit
```

Get Result

- ▶ You can check result from using `$ tail` or `$ more` (ex. `tail job_1_run` and `more job_1_run`).

```
1 | $ tail <output>
2 | $ more <output>
```

- ▶ You may have a line of code in your file (ex. `testpython.py`) to export your results as a CSV file or other formats.

Policies

Policies



Any individual or group wishing to use the **KVIS Mini-HPC Cluster (BistKA)** must submit an internal request by completing and uploading form (FM-HPC-01, FM-HPC-02, FM-HPC-03A or FM-HPC-03B).

All users must strictly comply with all policies outlined below.

Purpose



The **KVIS Mini-HPC Cluster (BistKA)** may only be used for three purposes: **(1) Student Projects, (2) Teacher Projects, and (3) School Classes that require high-performance computing workloads (CPU/GPU, high-memory)**

Terminologies

1. **Student Project** : Academic research conducted by students that requires computational resources to complete tasks.
2. **Teacher Project** : Academic research conducted by teachers that requires computational resources to complete tasks.
3. **School Class** : A course offered under the KVIS curriculum that requires computational resources for instructional purposes.
4. **Project** : Refers to both Student Projects and Teacher Projects.
5. **Group** : A general term that refers to a Student Project, a Teacher Project, or a School Class.
6. **Job** : A unit of work submitted to the HPC cluster for execution.
7. **User Log-in Account** : Personal account. Examples include: s00345, t651009.
8. **SLURM Account** : An account used to submit jobs and track computing credits. Examples include: s00345, t651009, 25sfcs01, 25sebl01, 25t01, 25cma30102.
9. **Partition** : A category of compute resources that defines the maximum runtime and task priority

for submitted jobs.

10. **Walltime** : The maximum duration a job is allowed to run on a compute resource. Once this time limit is reached, the job will be terminated automatically.
11. **Feb 28** : Refers to February 28 in a common year, or February 29 in a leap year.
12. **Credit** : The amount of resource usage for jobs in SLURM.
13. **SHr** : A unit of credit, where 1 SHr is based on 1 hour of usage on 1 physical processor core.

Good Practices and Usage Guidelines

1. **You can connect to the master node via CMD, Zsh, Terminal, or any other shell platform.** We also recommend using platforms of programs from [Login & File Transfer](#) . **VS Code is not recommended, BistKA blocks all vscode-server processes started by Remote - SSH extension,** as it consumes a large amount of system resources when working with the cluster.
2. **Try not to run code directly on the master node (i.e., without submitting a job),** as it consumes the master node's resources and may cause delays or lag in the command-line interface.
3. Request only the resources you actually need, such as CPU cores, RAM, and walltime.
4. Regularly delete unnecessary files to reduce storage usage.
5. Never share your user log-in account password with others.
6. Always log out after finishing your work.

Access & Usage Policy



Only eligible students (those with a Project ID or enrolled in a computational class) and eligible teachers (who advise a student project, have informed the Deputy Director of Academic Affairs, or teach a computational class) can register to use the KVIS Mini-HPC Cluster (BistKA).

The access is granted via CLI using a username and password.

1. **Only students with a school-assigned Project ID or those enrolled in a computational class, and teachers who either advise a student project, have informed the Deputy Director of Academic Affairs, or teach a computational class,** are eligible to register for the use of the KVIS Mini-HPC Cluster (BistKA).
2. The cluster is for high-performance computing workloads (CPU/GPU, large-memory).
3. **All jobs must be directly related to an approved school project or coursework for a computational class;** personal or unrelated tasks are not permitted.
4. **The requests for student project modification to use the mini-HPC cluster** must be reviewed and processed in accordance with the school's procedures and regulations.
5. Project status must be re-verified every academic year on **Feb 28**.

6. On-campus access is available via the KVIS-VISTEC network; off-campus access requires a personal VPN connection.
7. **All users are authorized access by user log-in account and password via terminal CLI.**
8. The cluster has a monitoring system that records IP addresses and login activity. **Any activities that violate usage policies on the master (login) node will be reported automatically.**

Account Policy

1. User log-in Account follow the student/teacher ID format (e.g., `s00345` , `t651009`), and each account is associated with a specific group based on the registration request.
2. Grade 12 (M6) student accounts, including their personal storage, will be removed on **Feb 28**.

Password Policy

1. **Minimum length: 8 characters.**
2. **It must have:**
 - ▶ Uppercase letters (A-Z): at least 1 required
 - ▶ Lowercase letters (a-z): at least 1 required
 - ▶ Digits (0-9): at least 1 required
 - ▶ Special characters: at least 1 required (e.g., ! @ # \$ ^ & * ?)
3. Do not use dictionary words or common weak words.
4. Do not include your username (student/teacher ID) in any form.

Job Policy



Jobs are run on a first-come, first-served basis with possible backfilling for short jobs.

The master node is for login, file transfer, and code compilation only. All compute work (including Jupyter and ipykernel) must run inside SLURM jobs, and **any unauthorized or excessive resource usage on the master node will be terminated automatically.**

1. Jobs are scheduled on a first-come, first-served basis, subject to resource availability (CPU/GPU, memory) and the requested wall time. Short jobs may be backfilled.
2. All submitted jobs must specify both the partition and SLURM account.
3. All jobs must be run from the CLI; graphical user interfaces are not supported on this system.
4. The master node may be used only for logging in to submit jobs, upload/download files, and compile programs. **Any process that exceeds the allowed CPU cores, memory (RAM), or runtime will be terminated automatically.**

5. All computational workloads—including Jupyter Notebook/Lab sessions and any ipykernel processes—must run inside SLURM jobs only. **Unauthorized or non-SLURM compute usage on the master node may be stopped without prior notice.**

6. Default resources:

Resource Type	Default Value
Partition	compute-devel
RAM	1 GiB
CPU-core	2 CPU-core
Walltime	30 mins

7. Maximum resources

Resource Type	Limitation
Job Submission	5 jobs/group
Job Running	2 jobs/group
CPU-cores	32 CPU-cores/group
GPUs	2 GPUs/group

8. Maximum walltime by partitions

Partition	Walltime	Priority	Remark
compute-normal	2 Days	1000	
compute-long	4 Days	500	
compute-devel	6 Hrs	1500	Default Partition
gpu4500-normal	2 Days	1000	
gpu4500-long	4 Days	500	
gpu4500-devel	6 Hrs	1500	

gpu4000-normal	2 Days	1000	<i>Currently not available</i>
gpu4000-long	4 Days	500	<i>Currently not available</i>
gpu4000-devel	6 Hrs	1500	<i>Currently not available</i>

Credit Policy



Each user, project, and class is given an initial SHr credit: **500 SHr per user account, 10,000 SHr per project, and 2,500 SHr per school class.**



If your project or school class requires additional job credits, **you must submit an Additional Project Resources Request Form (FM-HPC-05).**



The job credit quota for the user log-in account are not granted top-up.

1. **Each user log-in account starts with 500 SHr . Each project starts with 10,000 SHr . Each school class starts with 2,500 SHr**
2. If a project or school class requires more credit, please submit an internal request (FM-HPC-05).
3. Project credit expires on **Feb 28** of the academic year.
4. Class credit for the first semester expires on **Oct 31**.
5. Class credit for the second semester expires on **Mar 31**.
6. The student project credit for all Grade 12 will be expired on **Feb 28**, or on **May 31** with a date extension.
7. Hourly rate (SHr) by device

Resource Types	Credit (SHr) Usage
CPU	1.0 SHr/Core
RAM	0.1 SHr/GiB
RTX PRO 4500 GPU (32 GiB)	7.0 SHr/Card
RTX PRO 4000 GPU (24 GiB)	5.0 SHr/Card

Therefore, the **Total credit (SHr) usage per job** is calculated as:

$$\text{SHr} = N_{\text{nodes}} \times [(\alpha_{\text{cpu}} N_{\text{cores}}) + (\alpha_{\text{mem}} \text{Mem}) + (\alpha_{4500\text{gpu}} N_{\text{gpus}}) + (\alpha_{4000\text{gpu}} N_{\text{gpus}})] \times T_{\text{exec}}$$

where

T_{exec} : Execution time in hours,

N_{core} : Number of CPU cores allocated per node,

Mem : Memory size allocated per node, in GiB,

$N_{4500\text{gpu}}$: Number of RTX PRO 4500 GPUs allocated per node,

$N_{4000\text{gpu}}$: Number of RTX PRO 4000 GPUs allocated per node,

$\alpha_{\text{cpu}} (= 1.0)$: Weight per CPU processor core,

$\alpha_{\text{mem}} (= 0.1)$: Weight of memory size per GiB,

$\alpha_{4500\text{gpu}} (= 7.0)$: Weight per RTX PRO 4500 GPU accelerator,

$\alpha_{4000\text{gpu}} (= 5.0)$: Weight per RTX PRO 4000 GPU accelerator.

Disk Policy



Each user gets **50 GiB in their home directory**, and each project or school class gets **300 GiB per project/class folder**.



If your project or school class requires additional disk space, **you must submit an Additional Project Resources Request Form (FM-HPC-05)**.



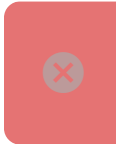
The disk quota for the user log-in account are not granted extension.

Disk space on the BistKA is a limited shared resource. Additional disk quota will be approved only when clearly justified, and may be granted at a lower size than requested depending on available capacity. **Please clean up unnecessary files** (e.g. temporary, duplicated, or test data) and back up important data to external storage before requesting more disk quota.

1. Each user log-in account has a maximum of **50 GiB** in their home directory. Each project and school class has a maximum of **300 GiB** per ID folder.
2. If a project or school class requires more than 300 GiB, please submit an internal request (FM-HPC-05).

3. Project storage expires on **Feb 28** of the academic year.
4. Class storage for the first semester expires on **Oct 31**.
5. Class storage for the second semester expires on **Mar 31**.
6. The student project storage for all Grade 12 will be removed on **Feb 28**.

Expiry Date Extension



The project storage and credit will be removed on Feb 28. If you wish to continue the project, you must submit an expiry date extension form (FM-HPC-04) by **Feb 28** each year.

1. The student project storage and credit can be extended to **Feb 28** next academic year. However, for student projects with all Grade 12 (M6) student members, the storage and credit can be only extended to **May 31** this year.
2. The teacher project storage and credit can be extended to **Feb 28** next academic year.
3. No school class can be extended date.

Project Folder and School Class Folder

1. Project and school class directories are available at [/project](#) or [/lustrefs/project](#).
2. The Folder name:
 - ▶ Purposes:
 - ▶ **Student Project** : YYs(f/a)(department)XX | EX. [25sfma01](#) , [25seb101](#)
 - ▶ **Teacher Project** : YYtXX | EX. [25t01](#)
 - ▶ **School Class** : YYc(subject area)XXXXX | EX. [25cst30273](#) , [25cma30201](#)
 - ▶ Code format:
 - ▶ **YY** : last two digits of the year in Gregorian calendar, based on the academic year
 - ▶ **XX...X** : Some specific ID
 - ▶ **s** : Student project
 - ▶ **t** : Teacher project
 - ▶ **c** : School class
 - ▶ **f** : Final project
 - ▶ **e** : Extra project

Summary

	User Log-in Account	Group		
		Student Project	Teacher Project	School Class
Disk Space	50 GiB	300 GiB	300 GiB	300 GiB
Credit (SHr)	500 SHr	10,000 SHr	10,000 SHr	2,500 SHr
Expiry Date (Disk Space and Credit)	S: Feb 28 for Grade 12 T: Not expire	Feb 28	Feb 28	Sem1: Oct 31 Sem2: Mar 31
Extend Expiry Date	S: May 31 (Grade 12 only)	Feb 28 Next year or May 31 for Grade 12	Feb 28 Next year	No
More Disk Space or Credit	No	Yes	Yes	Yes

How to Register

How to Register

Who can register?

Access to the **KVIS Mini-HPC Cluster (BistKA)** is limited to:

- ▶ **Students** who:
 - ▶ Have a school-assigned Project ID (student project), or
 - ▶ Are enrolled in a computational class.
- ▶ **Teachers** who:
 - ▶ Advise a student project,
 - ▶ Have informed the Deputy Director of Academic Affairs about their project, or
 - ▶ Teach a computational class that requires HPC.

The cluster may be used only for **(1) Student Projects**, **(2) Teacher Projects**, and **(3) School Classes**, that require high-performance computing workloads (CPU/GPU, high-memory). Personal or unrelated work is not permitted.



All users must agree to follow the **BistKA Policies**, including Access & Usage, Account, Password, Job, Credit, and Disk policies.

Choose the Correct Request Form

To register, you must submit an internal request using one of the [FM-HPC forms](#).

Form ID	Purpose / Who should use it
FM-HPC-01	Student Project Request Form – for student research projects.
FM-HPC-02	Teacher Project Request Form – for teacher research projects.

FM-HPC-03A	School Class Request Form – for KVIS courses that use the cluster.
FM-HPC-03B	Class's Student Attendees List Form – to list all students in a class.

Step-by-step Registration Process

Step 1 – Read the BistKA Policies

Before registering, carefully read the [policies](#) page to understand:

- ▶ What the cluster can be used for (Project/Class only)
- ▶ Access & Usage, Account, Password, Job, Credit, and Disk policies



All users must acknowledge the KVIS Mini-HPC [policies](#) and agree to comply with all rules and regulations.

Step 2 – Prepare Project or Class Information

Collect the information required by the form, such as:

- ▶ Project or class title
- ▶ School-assigned project ID or course ID (if already assigned)
- ▶ Project's School Advisors name and contact
- ▶ Class's student attendees

This information will be used to create **user log-in accounts**, **SLURM accounts**, **project folders**, and **initial job credit and disk quota**.



Note: Please check required information from the request form.

Step 3 – Download, Complete, and Upload FM-HPC form(s)

(1) From the Request Forms (FM-HPC) page, download the correct form

- ▶ **Student project** (FM-HPC-01)
- ▶ **Teacher project** (FM-HPC-02)
- ▶ **School class** (FM-HPC-03A)

► **Class student list** (FM-HPC-03B)

Fill in all required fields. Follow any instructions in the form (signatures, approvals, etc.)

(2) Fill and sign the PDF/Doc as required (student/teacher and, if needed, supervisor or Deputy Director of Academic Affairs).

(3) Go back to the [Request Forms \(FM-HPC\) & Submission](#) and click the MS Form upload link for the corresponding FM-HPC form.

(4) Upload the completed form file(s) and provide any additional information requested in the MS Form.

Step 4 – Administrative Review and Approval

The BistKA support/administration team will:

(1) Verify that you are eligible (student/teacher and project/class type) based on [policies](#).

(2) Check that the use case matches the allowed purposes (student/teacher project or school class).

(3) Allocate an appropriate **SLURM account**, **project/class folder**, and **initial credits and disk quota** according to the Credit and Disk Policies:

- **500 SHr** per user log-in account
- **10,000 SHr** per student/teacher project
- **2,500 SHr** per school class
- **50 GiB** in each user's home directory and 300 GiB per project/class folder.



Note: If anything is missing or unclear, the team may contact you for clarification.

Step 5 – Receive Your Login Information

Once your request is approved:

- A **user log-in account** (e.g. s00345, t651009) is created or activated for you.
- One or more SLURM accounts (for the project or class) are created and linked to your user account for job credit usage.
- **Your initial username and default password**, along with any project/class IDs and basic usage instructions, will be sent to you by KVIS email.



Note: On your first login, you must change your password to comply with the BistKA Password Policy (minimum 8 characters, including uppercase, lowercase, digit, and special character; no dictionary or weak words; do not include your ID in the password).

Step 6 – Start using BistKA

After registration and receive initial login via KVIS email:

- ▶ Connect to **BistKA** via a supported terminal/SSH client (e.g. Tabby, Termius, iTerm2, MobaXterm). **For detailed instructions, please refer to [Getting Started](#) and [Login & Transfer](#).**
- ▶ Check your initial disk quota and job credit using:

```
1 | $ myquota
2 | $ mycredit
```

Confirm you can see:

- ▶ **\$ myquota** for disk quota
 - ▶ You have **50 GiB** for **home directory**.
 - ▶ If you have any assigned project/class folder(s) under **/lustrefs/project**, you will have **300 GiB**.
- ▶ **\$ mycredit** for job credit quota
 - ▶ You have **500 SHr** for user log-in account.
 - ▶ If you have any assigned project(or class) folder(s) under **/lustrefs/project**, you will have **10000 (or 2500) SHr**.

Basic Commands

Basic Commands

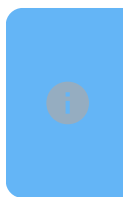
The BistKA Mini-HPC Cluster runs under Linux distribution, [Rocky Linux 9.6 \(Blue Onyx\)](#) . These below are **Linux commands** for the system, executed in a **Bash shell** or **terminal**.

Linux Commands

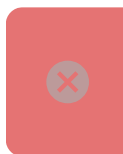
Fundamental commands for file and system operations

Command	Description	Example(s)
<code>\$ ls</code>	Lists files and directories in the current directory.	<code>\$ ls</code>
<code>\$ pwd</code>	Prints the current working directory.	<code>\$ pwd</code>
<code>\$ mkdir</code>	Creates a new directory (folder).	<code>\$ mkdir</code> <code>\$ mkdir folder</code>
<code>\$ cd</code>	Changes the current working directory to the specific folder.	<code>\$ cd folder</code> <code>\$ cd /lustrefs/project</code>
<code>\$ rm</code>	Removes files or directories (<code>-r</code> for recursive).	<code>\$ rm text.txt</code> <code>\$ rm -r folder</code>
<code>\$ cp</code>	Copies files or directories.	<code>\$ cp file.txt file_copy.txt</code>
<code>\$ mv</code>	Moves or renames files or directories.	<code>\$ mv old.txt new.txt</code>

<code>\$ cat</code>	Displays the contents of a file.	<code>\$ cat file.txt</code>
<code>\$ vi</code>	Opens the text editor to view or edit a file.	<code>\$ vi file.txt</code> <code>\$ vi script.sub</code>
<code>\$ tail</code>	Displays the last few lines of a file (default 10 lines).	<code>\$ tail log.txt</code>



In the Bash shell or terminal, you can copy or paste text by right-clicking and selecting **Copy** or **Paste**. Press **Ctrl + C** to stop (terminate) the current process, and **Ctrl + Z** to suspend (pause) it.



The cluster cannot restore deleted data, so please consider carefully before deleting anything.

HPC Commands

Specific commands for job submission, monitoring, and quota checking in the HPC environment

Command	Description	Example(s)	Remark
<code>\$ sbatch</code>	Submit job.	<code>\$ sbatch script.sub</code>	The file must be \$ bash format
<code>\$ sinfo</code>	Check partition and its own policy.	<code>\$ sinfo</code>	
<code>\$ scancel</code>	Cancel job <JOBID> by single job, multiple jobs, and all jobs under user-login account	<code>\$ scancel 99</code> <code>\$ scancel 101 102</code>	

		<code>\$ scancel -u <username></code>	
<code>\$ sinteractive</code>	Submit job via interactive mode.	<code>\$ sinteractive</code>	
<code>\$ myquota</code>	Check disk quota.	<code>\$ myquota</code>	
<code>\$ myqueue</code>	Check job status under user log-in account.	<code>\$ myqueue</code>	
<code>\$ squeue</code>	Check all job status.	<code>\$ squeue</code>	
<code>\$ mycredit</code>	Check job credit.	<code>\$ mycredit</code>	

Login & File Transfer

Login & File Transfer

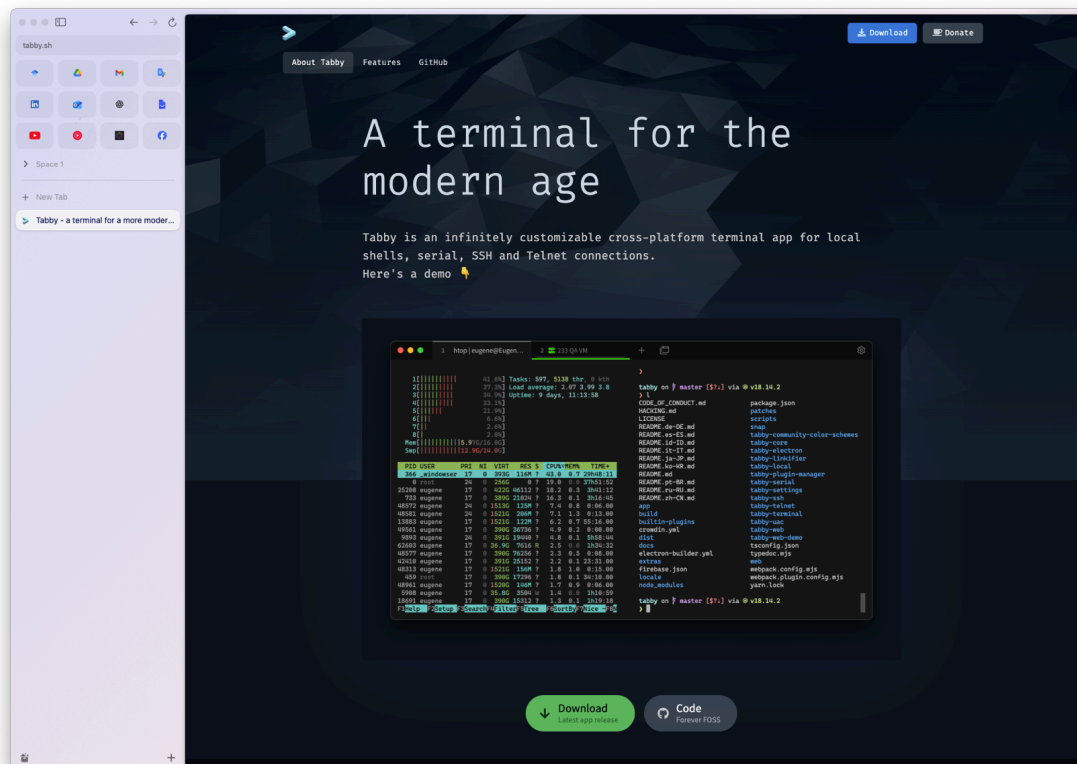
To login **BistKA** , users will receive their **BistKA** login credentials (user log-in account/ **username** and default **password**) via email. On the first login, you are required to change your password immediately. The new password must be at least **8 characters** long and include at **least one uppercase letter, one lowercase letter, one digit, and one special character**.

For connecting to BistKA, we recommend using **Tabby (tabby.sh)** , a free and open-source, modern, cross-platform terminal and SSH client available for Windows, macOS, and Linux.

Login to BistKA Mini-HPC Cluster

Install Tabby

1. Go to the Tabby website: <https://tabby.sh> 
2. Download the installer for your operating system (Windows, macOS, or Linux).
 - ▶ Windows: **tabby-*-setup-x64.exe** or **tabby-*-setup-arm64.exe**
 - ▶ macOS: **tabby-*-macos-x86_64.dmg** or **tabby-*-macos-arm64.dmg**
 - ▶ Linux: **tabby-*-linux-x64.*** or **tabby-*-linux-arm64.*** (Based on the Linux distribution of your system)

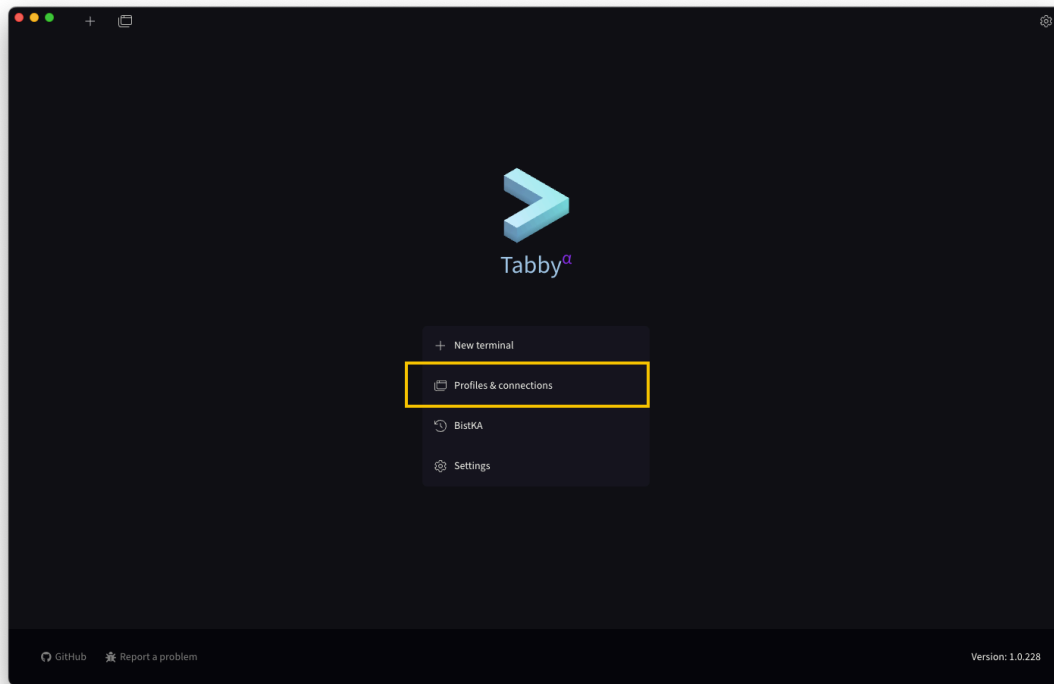


3. Install Tabby using the standard installer for your OS.
4. Launch Tabby.

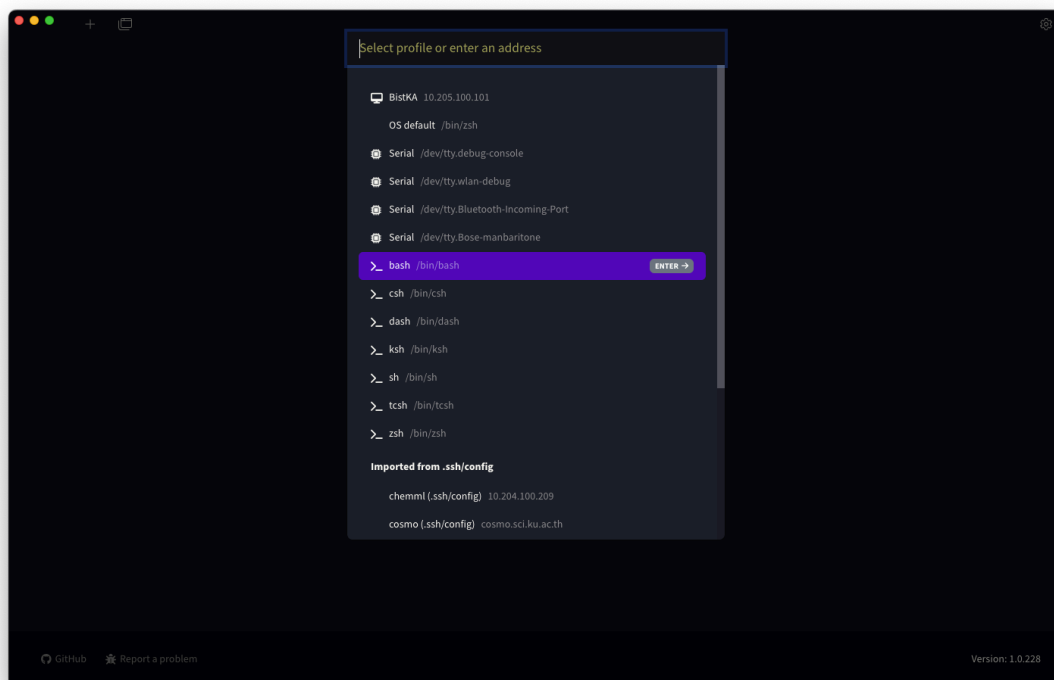
First Log in to BistKA

This process you need to set new password for **BistKA** via **bash** profile.

1. Click **Profiles & connection**.



2. Select **bash** profile and enter.



3. Type **ssh <username>@10.205.100.101** and enter.

4. Enter default password that received from email.

5. You will see the password policy. First, type default password for **Current password**, and then type your the new password and retype new password again to check password matching.


```
1 ssh hpcuser01@10.205.100.101  +  [ ]
ssh hpcuser01@10.205.100.101
hpcuser01@10.205.100.101's password:
You are required to change your password immediately (administrator enforced).
You are required to change your password immediately (administrator enforced).
WARNING: Your password has expired.
You must change your password now and login again!
Changing password for user hpcuser01.

[BistKA Password Policy]

* Minimum length: 8 characters.
* It must have:
  * Uppercase letters (A-Z): at least 1 required
  * Lowercase letters (a-z): at least 1 required
  * Digits (0-9): at least 1 required
  * Special characters: at least 1 required (e.g., ! @ # $ ^ & * ?)
* Do not use dictionary words or common weak words.
* Do not include your username in any form.

Current password:
New password:
Retype new password: 
```

6. If password is matching and with through the password policy you will get “passwd: all authentication tokens update succesfully.” and then the system will automatically exit.
7. You can try to login **BistKA** with **bash** again by using same user-login account and new password. You will get welcome message.

[illegible]

```

# Software folder: /apps
# Modules/Packages list: $ml av
#
# BistKA Helpdesk: hpc@kvis.ac.th
# (Please see guideline via website for how to submit support service ticket)
#
#####
Announcement:
* During this period, while the system is being tested, the cluster may not operate optimally.

Policy:
* All job execution must be submitted via job scheduler
* Initial Credit (SHr) for Job Submission
  User Credit: 500 SHr
  Project Credit: 10000 SHr
* Job submission
  Maximum Job Submission: 5 jobs/group
  Maximum Job Running: 2 jobs/group
  Maximum CPU-cores = 32 CPU-cores/group
  Maximum Walltime by Partition
    - compute-normal: 2 days | compute-long: 4 days | compute-devel: 6 hours
  Default CPU-core|RAM|Walltime: 1 CPU-cores|1 GB|30 minutes

Basic Commands for Job and Storage:
* Submit Job: $sbatch
* Check Partition and its Own Policy: $sinfo
* Cancel Job: $scancel <jobid>
* Submit Job via Interactive Mode: $sinteractive
* Check Disk Quota: $myquota
* Check Job Status: $myqueue
* Check Credit: $mycredit

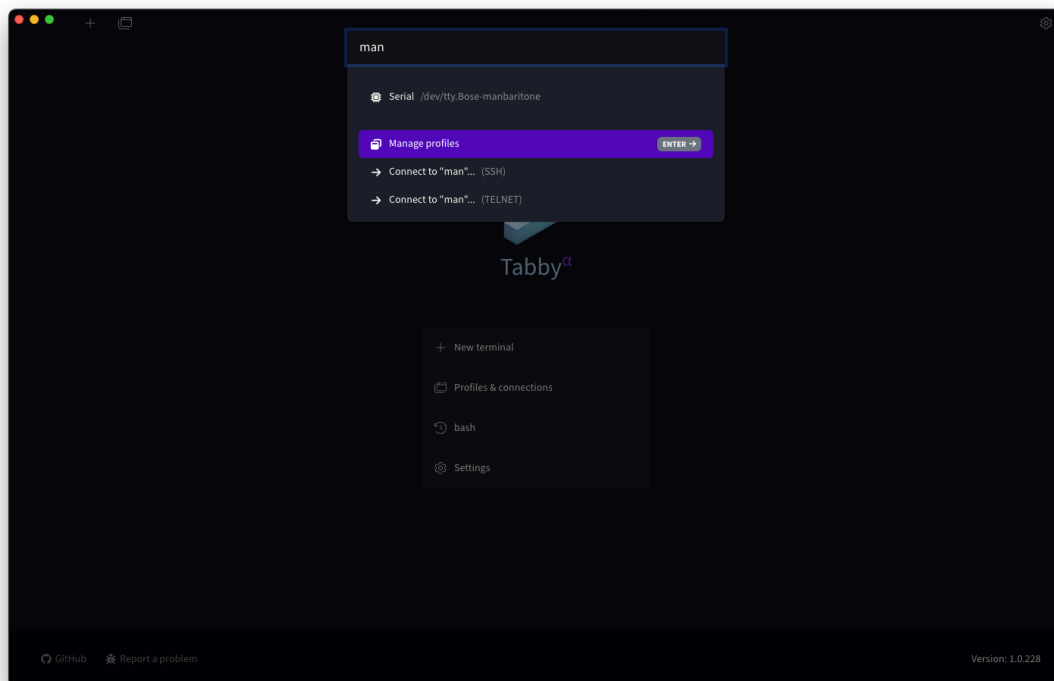
[hpcuser01@bistka-master ~]$

```

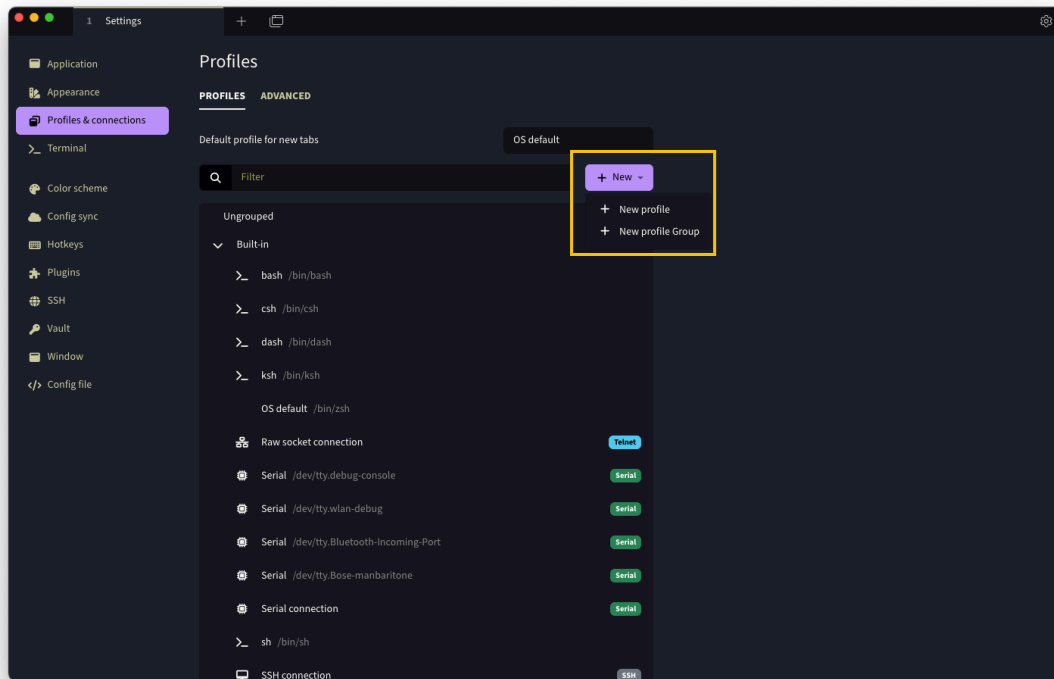
Create a BistKA SSH Profile

This process you only need to do this once; afterwards you just double-click to connect.

1. Open Tabby.
2. Click **Profiles & connection** and then select **Manage profiles**.

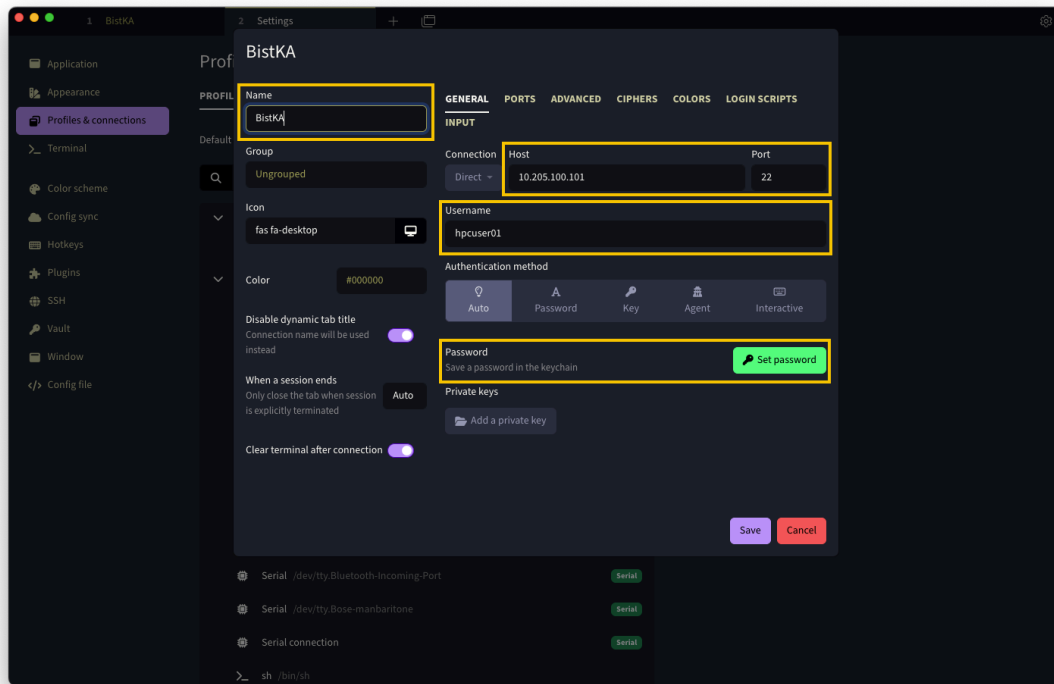


3. Click **+ New** button then click **New profile** and select **SSH Connection**.



4. Enter following
- Name:** BistKA
 - Host:** 10.205.100.101
 - Port:** 22
 - Username:** Your User log-in Account

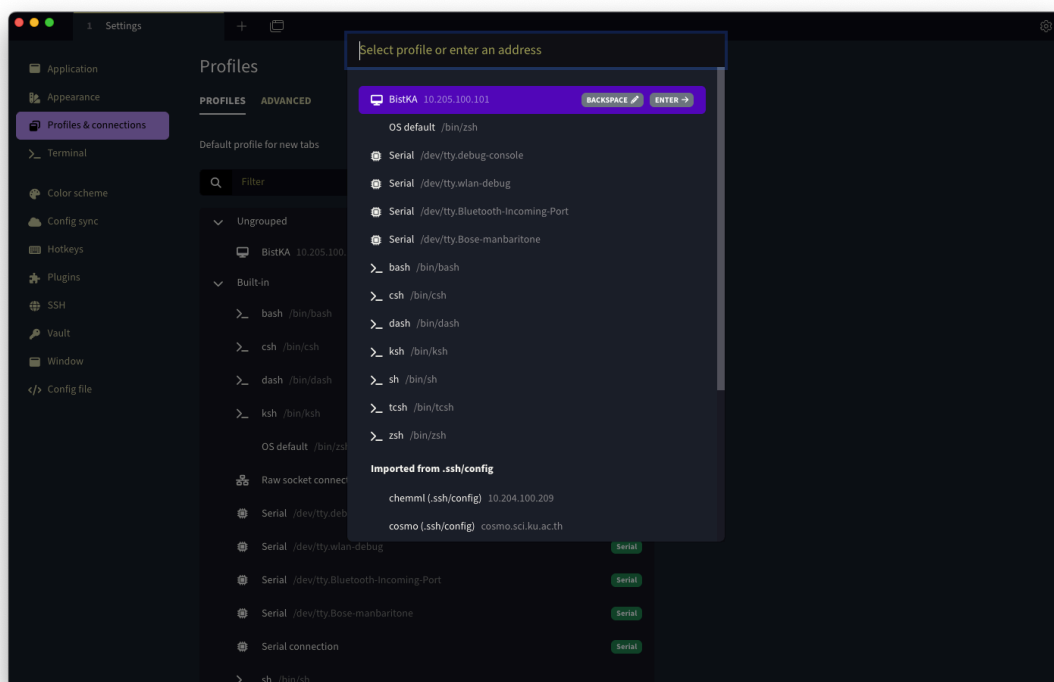
To save your password, click [Set Password](#) button and enter your password that previously set to save your password.



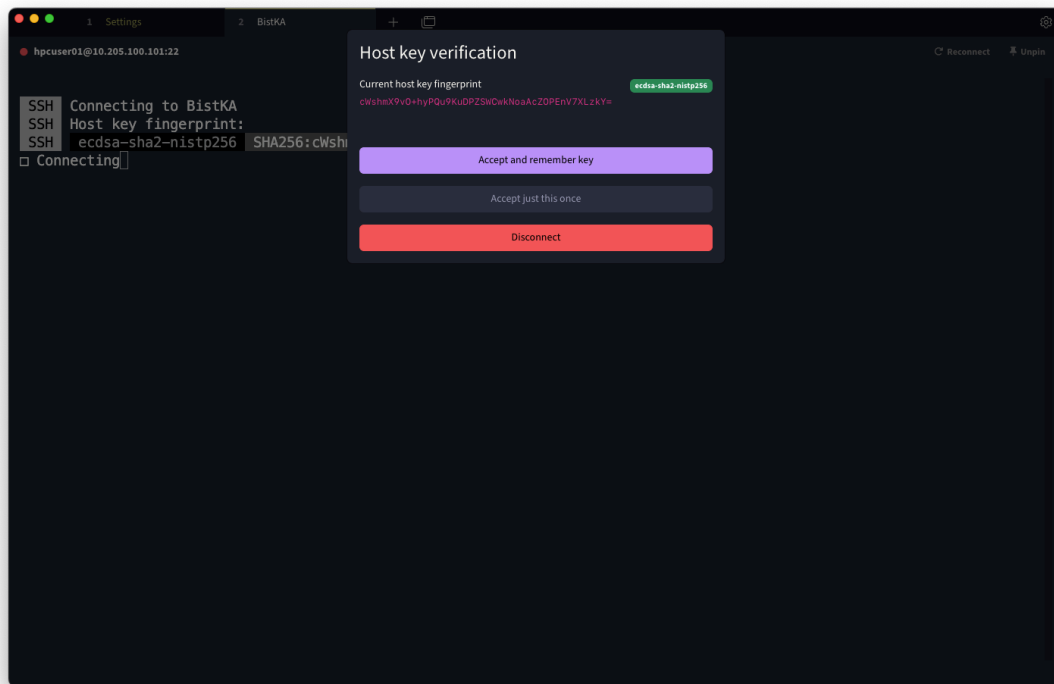
5. Click **Save**

Log in to BistKA via Tabby

1. Find **BistKA** profile at the **Profiles & connection** drop down and then click to login account.



2. For first time through **SSH** method, the system ask to verify your key fingerprint, click **Accept and remember key**.

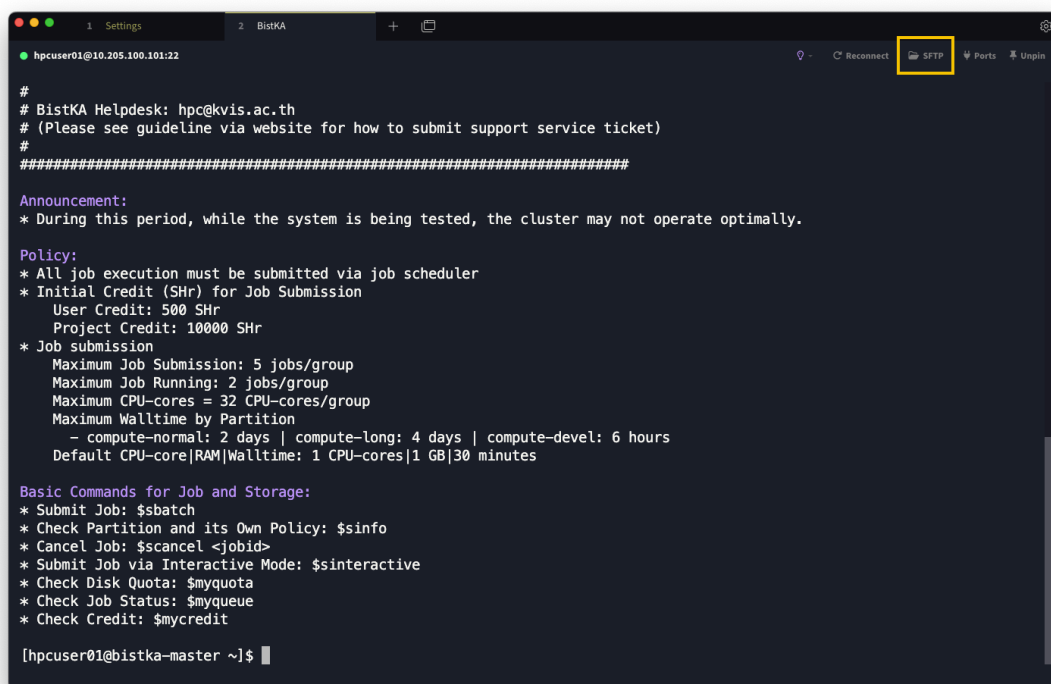


3. If success to login **bistka-master** host, you will see your log-in user account and welcome message.

File transfer using Tabby (SFTP panel)

Tabby provides a built-in SFTP file browser associated with each SSH connection, so you can drag-and-drop files between your computer and **BistKA**.

1. With your **BistKA** SSH session click **SFTP** at right conner of the window.



```

hpcuser01@10.205.100.101:22

#
# BistKA Helpdesk: hpc@kvis.ac.th
# (Please see guideline via website for how to submit support service ticket)
#
#####

Announcement:
* During this period, while the system is being tested, the cluster may not operate optimally.

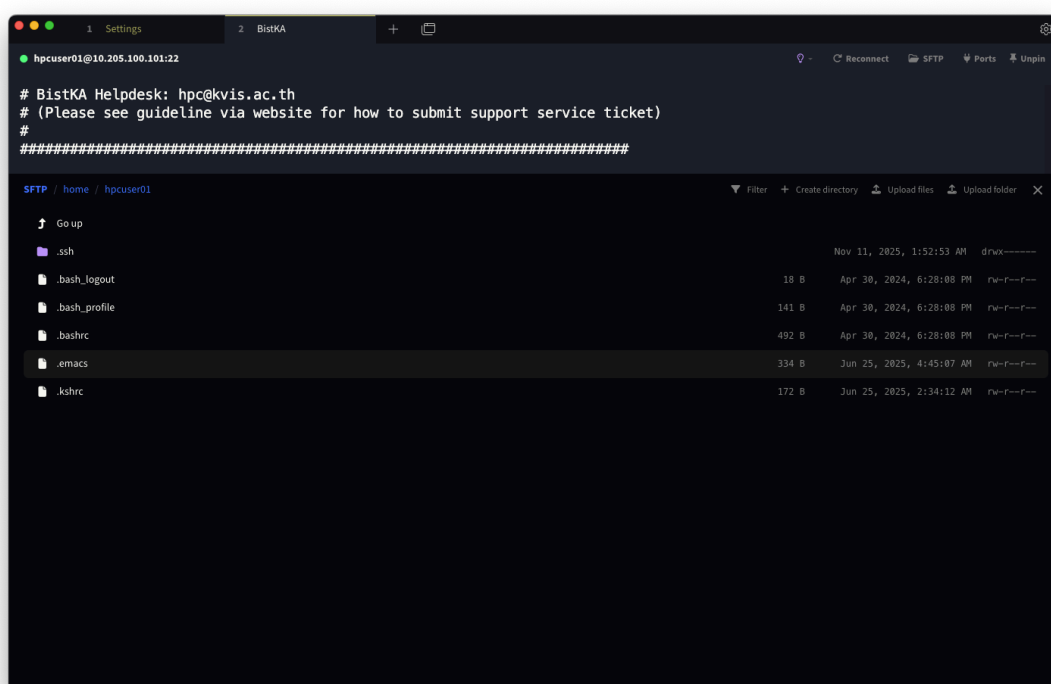
Policy:
* All job execution must be submitted via job scheduler
* Initial Credit (SHr) for Job Submission
  User Credit: 500 SHr
  Project Credit: 10000 SHr
* Job submission
  Maximum Job Submission: 5 jobs/group
  Maximum Job Running: 2 jobs/group
  Maximum CPU-cores = 32 CPU-cores/group
  Maximum Walltime by Partition
    - compute-normal: 2 days | compute-long: 4 days | compute-devel: 6 hours
  Default CPU-core|RAM|Walltime: 1 CPU-cores|1 GB|30 minutes

Basic Commands for Job and Storage:
* Submit Job: $sbatch
* Check Partition and its Own Policy: $sinfo
* Cancel Job: $scancel <jobid>
* Submit Job via Interactive Mode: $sinteractive
* Check Disk Quota: $myquota
* Check Job Status: $myqueue
* Check Credit: $mycredit

[hpcuser01@bistka-master ~]$

```

2. A new file browser panel opens showing the remote filesystem on **BistKA** . Go to **\$HOME** directory at **/home/<username>** or **/lustrefs/home**



```

SFTP / home / hpcuser01
Filter + Create directory Upload files Upload folder X

Go up
.ssh Nov 11, 2025, 11:52:53 AM drwx-----
.bash_logout 18 B Apr 30, 2024, 6:28:08 PM rw-r--r--
.bash_profile 141 B Apr 30, 2024, 6:28:08 PM rw-r--r--
.bashrc 492 B Apr 30, 2024, 6:28:08 PM rw-r--r--
.emacs 334 B Jun 25, 2025, 4:45:07 AM rw-r--r--
.kshrc 172 B Jun 25, 2025, 2:34:12 AM rw-r--r--

```

3. To upload files or folder from your laptop/desktop to **BistKA** :
 - ▶ In the SFTP panel, navigate to the target directory on **BistKA** , usually:
 - ▶ **\$HOME** or **/lustrefs/home/<username>** for personal files, or

- ▶ A project directory under [/lustrefs/project/...](#) (if you have access).
 - ▶ From your local machine:
 - ▶ Drag the file from your desktop / file explorer into the SFTP window, or
 - ▶ Use the [Upload files](#) / [Upload folder](#) button in the SFTP panel and select the file/folder.
4. To download files or folder from [BistKA](#) to your computer:
- ▶ In the SFTP panel, navigate to the directory that contains your output files (e.g. \$HOME/results) on [BistKA](#) .
 - ▶ Click the file(s) you want to download and select path of your computer where you want to save the files, and then click [Save](#) .
 - ▶ For download folder, click right at the folder and then click [Download directory](#) – select path of your computer where you want to save the folder – and then click [Download Here](#) .

Alternative Software

1. Termius (Windows / macOS / Linux / iOS / Android) – Termius is a modern, cross-platform SSH client with a nice GUI and built-in SFTP browser. It supports SSH, SFTP, Mosh, Telnet, port forwarding, and syncs hosts/keys across devices via an encrypted cloud vault.

2. iTerm2 (macOS only) – iTerm2 is an advanced terminal emulator for macOS that replaces the default Terminal app. It adds split panes, a hotkey drop-down terminal, powerful search, and lots of quality-of-life features.

3. MobaXterm (Windows only) – MobaXterm is an all-in-one remote access tool for Windows. It bundles a tabbed terminal, SSH client, SFTP browser, and an X11 server so you can also run remote graphical applications.

Tool	OS	Built-in SFTP GUI	Notes
Tabby	Win / macOS / Linux	Yes	Good default choice, modern UI, easy SFTP.
Termius	Win / macOS / Linux / mobile	Yes	Great if you want sync across devices and nice host management.
iTerm2	macOS only	No (terminal)	Best for power terminal users; use scp/sftp or separate SFTP app.

		only)	
MobaXterm	Windows only	Yes	All-in-one SSH + SFTP + X11; very popular for HPC on Windows.

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

Module Utility

Module Utility

Introduction

On the [BistKA](#), many applications are provided through a module system based on [Lmod](#). In Linux, your “working environment” is defined by a collection of environment variables—named values that programs use to locate files, libraries, and configuration settings. Common examples include [\\$HOME](#), which points to your home directory, and [\\$PATH](#), which lists directories where the system looks for executable programs. While a default set of variables is created when you open a shell, this environment often needs to be adjusted for scientific software (for example, setting license variables or adding specialized compiler and MPI paths).

The module utility (commands `$ module` or `$ ml`) provides a clean way to manage this environment on [BistKA](#). Each installed software package (and often each version) is described by a modulefile that knows which environment variables to set or modify: paths to executables and libraries, required dependencies, and any additional configuration. By loading a module for a particular application, all relevant variables are automatically configured for you; by unloading it, those changes are cleanly removed. This allows users to switch between different software stacks and versions without manually editing their environment, ensuring that jobs run with the correct compilers, libraries, and tools on the [BistKA](#) system.

Default Modules

The default environment of [BistKA](#) is automatically loaded at login via [bistka](#) module.

```
1 | Currently Loaded Modules:
2 |   1) autotools   3) hwloc/2.12.0   5) ucx/1.18.0       7) numactl/2.0.19-
3 | GCCcore-14.2.0   9) openmpi5/5.0.7
   |   2) prun/2.2    4) gnu14/14.2.0   6) GCCcore/14.2.0   8)
   | libfabric/2.0.0-GCCcore-14.2.0 10) bistka
```

Available Modules

```

1  ----- /opt/ohpc/pub/moduledeps/gnu14-
2  openmpi5 -----
3  adios2 (1)  extrae (1)  otf2 (1)  pnetcdf (1)  scalasca (1)
4  tau (1)
5  boost (1)  imb (1)  petsc (1)  py3-mpi4py (1)  scorep (1)
6  trilinos (1)
7  dimemas (1)  omb (1)  phdf5 (1)  scalapack (1)  sionlib (1)
8
9  -----
10 /opt/ohpc/pub/moduledeps/gnu14 -----
11 ---
12 R (1)  gotcha (1)  likwid (1)  netcdf-cxx (1)  openblas (1)
13 plasma (1)  superlu (1)
14 cubelib (1)  hdf5 (1)  mpich (1)  netcdf (1)  openmpi5 (1)
15 py3-numpy (1)
16 cubew (1)  impi (1)  mvapich2 (1)  opari2 (1)  pdtoolkit (1)
17 scotch (1)
18
19 -----
20 /lustrefs/apps/easybuild/modules/all -----
21 -----
22 AOCL-BLAS (1)  GLib (4)  OpenSSL (2)  binutils (7)
23 intltool (4)  lz4 (4)
24 Abseil (3)  GMP (4)  PCRE2 (4)  bzip2 (4)
25 jbigkit (4)  make (3)
26 Autoconf (5)  GSL (2)  PMIx (3)  cURL (4)
27 libGLU (2)  maturin (1)
28 Automake (5)  HDF (3)  PRRTE (2)  cairo (3)
29 libarchive (4)  ncurses (7)
30 Autotools (5)  HDF5 (2)  Perl-bundle-CPAN (3)  cffi (1)
31 libdeflate (4)  netCDF (1)
32 BLIS (2)  ICU (4)  Perl (5)  cryptography (1)
33 libdrm (3)  numactl (4)
34 Bison (5)  Imath (3)  PnetCDF (2)  expat (4)
35 libevent (3)  patchelf (2)
36 Boost (4)  Java (2)  Python (5)  ffnvcodec (4)
37 libfabric (3)  pixman (4)
38 Brotli (4)  LAME (4)  RapidJSON (3)  flex (5)
39 libffi (4)  pkgconf (5)
40 CMake (4)  LAPACK (2)  Rust (4)  flit (2)
41 libglvnd (3)  protobuf-python (2)
42 Catch2 (2)  LLVM (3)  SDL2 (4)  fontconfig (4)

```

```

43 | libiconv      (4)  protobuf      (3)
44 | Cython      (2)  LibTIFF      (4)  SQLite      (4)  foss      (2)
45 | libidn2      (1)  psutil      (3)
   | DB          (1)  M4          (6)  SWIG      (4)  freetype  (4)
   | libjpeg-turbo (4)  scikit-build-core (1)
   | Doxygen     (4)  MPFR      (4)  ScaLAPACK (2)  gettext  (7)
   | libpciaccess (4)  scikit-build (2)
   | Eigen       (2)  Mako      (4)  Szip      (4)  gfbf     (1)
   | libpng      (4)  setuptools-rust (1)
   | FFTW.MPI    (2)  Mesa      (2)  Tcl       (4)  git      (4)
   | libpsl      (1)  util-linux (4)
   | FFTW        (4)  Meson     (4)  UCC       (3)  glslang-SPIRV (1)
   | libreadline (4)  virtualenv (1)
   | FFmpeg      (4)  Miniforge3 (2)  UCX       (3)  gomp     (2)
   | libtirpc     (3)  x264      (4)
   | FLTK        (2)  NASM      (4)  UnZip     (4)  gperf    (4)
   | libtool      (5)  x265      (4)
   | FlexiBLAS   (2)  Nextflow  (3)  Wayland   (3)  groff    (4)
   | libunistring (1)  xorg-macros (4)
   | FriBidi     (4)  Ninja     (4)  X11       (4)  gzip     (4)
   | libunwind    (4)  xprop     (3)
   | GCC         (4)  OpenBLAS  (3)  XZ        (4)  hatchling (2)
   | libxml2     (4)  xxd       (2)
   | GCCcore     (4)  OpenJPEG  (4)  Yasm      (4)  help2man  (4)
   | libyaml     (1)  zlib      (7)
   | GLPK        (4)  OpenMPI   (3)  Z3        (2)  hwloc     (4)
   | lit         (4)  zstd      (4)

```

```

----- /opt/ohpc/pub/scientific
-----

```

```

ambertools  (1)  charmm (1)  dock6  (1)  gromacs  (1)
modeller (1)  openbabel (1)  pmemd  (1)
autodock-gpu (1)  cp2k  (1)  gamess (1)  lammps  (1)  namd
(2)  openfoam (1)  pythia  (1)
autodock-vina (1)  dbcsr  (1)  geos  (1)  mayachemtools (1)  octave
(1)  orca  (1)  quantumespresso (2)

```

```

----- /opt/ohpc/pub/modulefiles
-----

```

```

EasyBuild (1)  cmake (1)  gnu13 (1)  intel  (3)  os  (1)  prun
(1)  valgrind (1)
autotools (1)  cuda  (3)  gnu14 (1)  libfabric (1)  papi (1)  spack
(1)
bistka  (1)  gnu12 (1)  hwloc (1)  nvhpc  (3)  pmix (1)  ucx
(1)

```

Useful Commands

The module utility is invoked by the command module. This command must be followed by an instruction of what action is required and by the details of a pre-defined module.

The utility displays a help menu by doing

```

1 | $ ml help
2 |
3 | Usage: module [options] sub-command [args ...]
4 |
5 | Options:
6 |   -h -? -H --help           This help message
7 |   -s availStyle --style=availStyle Site controlled avail style: system
8 |   (default: system)
9 |   --regression_testing      Lmod regression testing
10 |  -b --brief                  brief listing with only user
11 | specified modules
12 |   -D                         Program tracing written to stderr
13 |   --debug=dbglvl            Program tracing written to stderr
14 |   (where dbglvl is a number 1,2,3)
15 |   --pin_versions            When doing a restore use specified
16 |   version, do not follow defaults
17 |   -d --default              List default modules only when used
18 |   with avail
19 |   ...

```

The utility displays the overview of modules by issuing the command

```

1 | $ ml ov
2 |
3 | ----- /opt/ohpc/pub/moduledeps/gnu14-
4 | openmpi5 -----
5 | adios2 (1)  extrae (1)  otf2 (1)  pnetcdf (1)  scalasca (1)
6 | tau (1)
7 | boost (1)  imb (1)  petsc (1)  py3-mpi4py (1)  scorep (1)
8 | trilinos (1)
9 | dimemas (1)  omb (1)  phdf5 (1)  scalapack (1)  sionlib (1)
10 |
11 | -----

```

```

12 /opt/ohpc/pub/moduledeps/gnu14 -----
13 ---
14 R      (1)  gotcha (1)  likwid   (1)  netcdf-cxx (1)  openblas  (1)
15 plasma  (1)  superlu (1)
16 cubelib (1)  hdf5   (1)  mpich   (1)  netcdf    (1)  openmpi5  (1)
17 py3-numpy (1)
18 cubew   (1)  impi   (1)  mvapich2 (1)  opari2    (1)  pdtoolkit (1)
19 scotch  (1)
20
21 -----
22 /lustrefs/apps/easybuild/modules/all -----
23 -----
24 AOCL-BLAS (1)  GLib      (4)  OpenSSL      (2)  binutils     (7)
25 intltool   (4)  lz4          (4)
26 Abseil     (3)  GMP          (4)  PCRE2        (4)  bzip2        (4)
27 jbigkit    (4)  make         (3)
28 Autoconf   (5)  GSL          (2)  PMIx         (3)  cURL         (4)
29 libGLU     (2)  maturin      (1)
30 Automake   (5)  HDF          (3)  PRRTE        (2)  cairo        (3)
31 libarchive (4)  ncurses      (7)
32 Autotools  (5)  HDF5         (2)  Perl-bundle-CPAN (3)  cffi         (1)
33 libdeflate (4)  netCDF       (1)
34 BLIS       (2)  ICU          (4)  Perl         (5)  cryptography (1)
35 libdrm     (3)  numactl      (4)
36 Bison      (5)  Imath        (3)  PnetCDF      (2)  expat        (4)
37 libevent   (3)  patchelf     (2)
38 Boost      (4)  Java         (2)  Python       (5)  ffnvcodec    (4)
39 libfabric  (3)  pixman       (4)
40 Brotli     (4)  LAME         (4)  RapidJSON    (3)  flex         (5)
41 libffi     (4)  pkgconf      (5)
42 CMake      (4)  LAPACK       (2)  Rust         (4)  flit         (2)
43 libglvnd   (3)  protobuf-python (2)
44 Catch2     (2)  LLVM         (3)  SDL2         (4)  fontconfig   (4)
45 libiconv   (4)  protobuf     (3)
46 Cython     (2)  LibTIFF      (4)  SQLite       (4)  foss         (2)
47 libidn2    (1)  psutil       (3)
   DB        (1)  M4           (6)  SWIG         (4)  freetype     (4)
   libjpeg-turbo (4)  scikit-build-core (1)
   Doxygen     (4)  MPFR         (4)  ScaLAPACK    (2)  gettext      (7)
   libpciaccess (4)  scikit-build   (2)
   Eigen       (2)  Mako         (4)  Szip         (4)  gfbf         (1)
   libpng      (4)  setuptools-rust (1)
   FFTW.MPI    (2)  Mesa         (2)  Tcl          (4)  git          (4)
   libpsl      (1)  util-linux    (4)
   FFTW        (4)  Meson        (4)  UCC          (3)  glslang-SPIRV (1)

```

```

libreadline (4)  virtualenv (1)
FFmpeg (4)  Miniforge3 (2)  UCX (3)  gomp (2)
libtirpc (3)  x264 (4)
FLTK (2)  NASM (4)  UnZip (4)  gperf (4)
libtool (5)  x265 (4)
FlexiBLAS (2)  Nextflow (3)  Wayland (3)  groff (4)
libunistring (1)  xorg-macros (4)
FriBidi (4)  Ninja (4)  X11 (4)  gzip (4)
libunwind (4)  xprop (3)
GCC (4)  OpenBLAS (3)  XZ (4)  hatchling (2)
libxml2 (4)  xxd (2)
GCCcore (4)  OpenJPEG (4)  Yasm (4)  help2man (4)
libyaml (1)  zlib (7)
GLPK (4)  OpenMPI (3)  Z3 (2)  hwloc (4)
lit (4)  zstd (4)

```

```
----- /opt/ohpc/pub/scientific
```

```

ambertools (1)  charmm (1)  dock6 (1)  gromacs (1)
modeller (1)  openbabel (1)  pmemd (1)
autodock-gpu (1)  cp2k (1)  gamess (1)  lammps (1)  namd
(2)  openfoam (1)  pythia (1)
autodock-vina (1)  dbcsr (1)  geos (1)  mayachemtools (1)  octave
(1)  orca (1)  quantumespresso (2)

```

```
----- /opt/ohpc/pub/modulefiles
```

```

EasyBuild (1)  cmake (1)  gnu13 (1)  intel (3)  os (1)  prun
(1)  valgrind (1)
autotools (1)  cuda (3)  gnu14 (1)  libfabric (1)  papi (1)  spack
(1)
bistka (1)  gnu12 (1)  hwloc (1)  nvhpc (3)  pmix (1)  ucx
(1)

```

or displays only the information related to a certain software package, e.g.

```

1 | $ ml av cmake
2 |
3 | -----
4 | /lustrefs/apps/easybuild/modules/all -----
5 | -----
6 |      CMake/3.24.3-GCCcore-12.2.0      CMake/3.27.6-GCCcore-13.2.0
7 | CMake/3.29.3-GCCcore-13.3.0      CMake/3.31.3-GCCcore-14.2.0 (D)

```

```

8 |
9 | -----
10 | -- /opt/ohpc/pub/modulefiles -----
11 | -----
12 |     cmake/4.0.0
13 |
14 | Where:
15 |     D:  Default Module
16 |
17 | If the avail list is too long consider trying:
18 |
    "module --default avail" or "ml -d av" to just list the default modules.
    "module overview" or "ml ov" to display the number of modules for each
    name.

    Use "module spider" to find all possible modules and extensions.
    Use "module keyword key1 key2 ..." to search for all possible modules
    matching any of the "keys".

```

The `$ ml av` instruction displays all the versions available for the installed applications, and shows which version is pre-defined as being the default. A software package is loaded with the load or the add instructions, e.g.

```

1 | $ ml gnu12

```

If no version is specified, the default version of the software is loaded. Specific versions, other than the default can be loaded by specifying the version, e.g.

```

1 | $ ml gnu12/12.2.0

```

The modules that are already loaded by users are displayed with the command

```

1 | $ ml
2 |
3 | Currently Loaded Modules:
4 |     1) gnu12/12.2.0

```

A module can be “unloaded” with the unload or rm instructions, e.g.

```
1 | ml unload gnu12
```

To unload all currently loaded modules in your session is commanded with

```
1 | ml purge
```

To reset default module in your session is commanded with

```
1 | ml bistka
```

If you try to load `boost/1.81.0` via `$ ml`, you will get

```
1 | $ ml boost/1.81.0
2 |
3 | Lmod has detected the following error: These module(s) or extension(s)
4 | exist but cannot be loaded as requested: "boost/1.81.0"
   | Try: "module spider boost/1.81.0" to see how to load the module(s).
```

and then

```
1 | $ module spider boost/1.81.0
2 | -----
3 | -----
4 | boost: boost/1.81.0
5 | -----
6 | -----
7 | Description:
8 |   Free peer-reviewed portable C++ source libraries
9 |
10 |
11 | You will need to load all module(s) on any one of the lines below
12 | before the "boost/1.81.0" module is available to load.
13 |
14 | gnu12/12.2.0  mpich/3.4.3-ofi
15 | gnu12/12.2.0  mvapich2/2.3.7
16 | gnu12/12.2.0  openmpi4/4.1.5
17 | gnu13/13.2.0  mpich/3.4.3-ofi
```



```
18 |      gnu13/13.2.0  mvapich2/2.3.7
19 |      gnu13/13.2.0  openmpi5/5.0.5
20 |      intel/2023.2.1 openmpi4/4.1.5
21 |      intel/2024.0.0 openmpi4/4.1.5
22 |      intel/2025.3.0 openmpi4/4.1.5
23 |
24 | Help:
25 |      This module loads the boost library built with the gnu12
      compiler toolchain and the mpich MPI stack.

      Version 1.81.0
```

Thus, if you need to load [boost/1.81.0](#) , you will command

```
1 | # ml gnu12 openmpi4 boost
```

To see all loaded modules

```
1 | $ ml
2 |
3 | Currently Loaded Modules:
4 |   1) gnu12/12.2.0   2) hwloc/2.12.0   3) ucx/1.18.0   4) libfabric/1.18.0
   5) openmpi4/4.1.5  6) boost/1.81.0
```

Software Catalog

Software Catalog



Each software package has its own licensing scheme. For example, some licenses restrict the physical location or machines where the software may be installed and used, some lock usage to a specific MAC address, and some require a license server to manage access. **We recommend that you carefully review the software's license agreement and ensure that your usage complies with the permitted terms.**

BistKA provides a curated catalog of HPC software via `Lmod` modules, covering molecular dynamics, quantum chemistry, CFD, and related toolchains. Load tools with `$ ml <module>` and run them inside `SLURM` jobs only (srun/sbatch); the master node is reserved for login, file transfer, job submission, and compiling. Where available, GPU-enabled builds are provided—request GPUs with `--gpus=<N>` and follow each tool's notes on MPI ranks and `OMP_NUM_THREADS`.

To discover software and versions, use `$ ml ov`, and to see environment details for a module, use `$ ml show <module>`. All software are available in the `/apps` or `/lustrefs/apps` folder.

1	----- /opt/ohpc/pub/scientific -----		
2	-----		
3	ambertools/25	dock6/6.13	modeller/10.8
4	orca/6.1.0-f.0		
5	autodock-gpu/1.6-cuda12	gamess/2024.2.1	namd/3.0.2-cuda
6	pmemd/24		
7	autodock-vina/1.2.7	geos/3.14.1	namd/3.0.2 (D)
	pythia/8.316		
	charmm/c50b1	gromacs/2025.3	octave/10.3.0
	quantumESPRESSO/7.5-cuda-12.8		
	cp2k/2025.2	lammps/22Jul2025	openbabel/3.1.1
	quantumESPRESSO/7.5	(D)	
	dbcsr/2.8.0	mayachemtools/061125	openfoam/13.0

Available Software

Software	Version	Module to load	CPU/GPU	What it's for
AmberTools	25	<code>\$ ml ambertools/25</code>	CPU / GPU	Amber prep/analysis, <code>tleap</code> , <code>cppt</code> parameterization conversion.
AutoDock-GPU	1.6	<code>\$ ml autodock-gpu/1.6-cuda12</code>	GPU	GPU-accelerated ligand-receptor docking.
AutoDock Vina	1.2.7	<code>\$ ml autodock-vina/1.2.7</code>	CPU	Fast docking on CPU.
CHARMM	c50b1	<code>\$ ml charmm/c50b1</code>	CPU	MD, minimization analysis across biomolecular workflows.
CP2K	2025.2	<code>\$ ml cp2k/2025.2</code>	CPU	Atomistic simulation (DFT/MD/MM); uses DBCSR for sparse algebra.
DBCSR	2.8.0	<code>\$ ml dbcsr/2.8.0</code>	CPU	Library for block-sparse linear algebra (used by CP2K).
DOCK6	6.13	<code>\$ ml dock6/6.13</code>	CPU	Molecular docking for biomolecules.
GAMESS	2024.2.1	<code>\$ ml gamess/2024.2.1</code>	CPU	Ab initio/DFT quantum chemistry.
GEOS	3.14.1	<code>\$ ml geos/3.14.1</code>	CPU	Computational geometry library operations on plane geometric objects.

GROMACS	2025.3	<code>\$ ml gromacs/2025.3</code>	CPU / GPU	MD engine for biomolecules/ma
LAMMPS	22Jul2025	<code>\$ ml lammps/22Jul2025</code>	CPU	Classical MD with force fields & opt packages.
LAMMPS	22Jul2025	<code>\$ ml lammps/22Jul2025</code>	CPU	Classical MD with force fields & opt packages.
MayaChemTools	06Nov2025	<code>\$ mayachemtools/061125</code>	CPU	Toolkit for chemoinformatic molecular chemis
Modeller	10.8	<code>\$ modeller/10.8</code>	CPU	Protein Structure Modeling.
NAMD	3.0.2	<code>\$ ml namd/3.0.2</code>	CPU	High-performanc for biomolecules.
NWChem	7.3.1	<code>\$ ml nwchem/7.3.1</code>	CPU	Scalable quantum chemistry (HF/DFT/MP2/CC)
Octave	10.3.0	<code>\$ ml octave/10.3.0</code>	CPU	High-level language numerical computations.
OpenBabel	3.1.1	<code>\$ ml openbabel/3.1.1</code>	CPU	Cheminformatics toolkit for file for conversion, desc and basic molecu mechanics.
OpenFOAM	13.0	<code>\$ ml openfoam/13.0</code>	CPU	CFD solvers/utilit and meshing too
ORCA	6.1.0 f.0	<code>\$ ml orca/6.1.0-f.0</code>	CPU	Quantum chemis (DFT, ab initio, spectroscopy).
			CPU /	Amber MD engin (<code>pmemd.MPI</code> ,

PMEMD	24	<code>\$ ml pmemd/24</code>	GPU	<code>pmemd.cuda</code>) fast production M
Pythia	8.316	<code>\$ ml pythia/8.316</code>	CPU	Monte Carlo gen for high-energy collisions.
Quantum ESPRESSO (CUDA)	7.5 (CUDA 12.8)	<code>\$ ml quantumespesso/7.5- cuda-12.8</code>	GPU	GPU-accelerated build.
Quantum ESPRESSO	7.5	<code>\$ ml quantumespesso/7.5</code>	CPU	Plane-wave DFT (SCF/NSCF/phonc

Job Scheduler

Job Scheduler

On the [BistKA](#), users do not run heavy applications directly on the login (master) node. Instead, all workloads are sent as jobs to the **SLURM** scheduler, which assigns them to suitable compute nodes. Each job specifies both the actual task to perform (the program to execute, required input files, and output locations) and the resources it needs, such as number of CPU cores, memory size, GPU type and quantity, time limit, and target partition.

Jobs on [BistKA](#) are executed as batch jobs, meaning they run in the background without the user needing to stay logged in. A typical workflow is: the user logs in to the [BistKA](#) master node, prepares a **SLURM** job script (or uses `$ srun` / `$ sbatch` commands) that specifies the application and resource requests, then submits the job to the scheduler. **SLURM** places the job in a queue, matches it with available resources according to the cluster policies and priorities, and starts it on one or more compute nodes. The user can safely log out while the job is running and later log in again to check status and retrieve results.

SLURM Commands

The table below gives a short description of the most used SLURM commands.

Command	Description
<code>\$ sacct</code>	Reports detailed accounting information for jobs that have completed or changed state. Useful for checking final resource usage (CPU time, memory, exit code, start/end time, etc.) on BistKA.
<code>\$ salloc</code>	Interactively allocates resources for a job and starts a shell within that allocation. Typically used when you want to run <code>\$ srun</code> commands or debug interactively on the allocated compute node(s).
<code>\$ sbatch</code>	Submits a batch job script to SLURM for later execution. The script usually contains SLURM <code>#SBATCH</code> directives and one or more <code>\$ srun</code> commands to start the application. The job is queued and run when resources are available.
<code>\$</code>	Cancels jobs managed by SLURM. Can be used to stop a pending or running job

<code>scancel</code>	by JobID, or to cancel multiple jobs matching specific users, partitions, or other filters.
<code>\$ sinfo</code>	Shows the status of partitions and nodes in the cluster (up, down, drained, allocated, etc.). Supports many filtering and formatting options to view node health and availability on BistKA.
<code>\$ squeue</code>	Displays the status of jobs in the SLURM queue. By default, lists running jobs followed by pending jobs in priority order. Can be filtered by user, partition, state, or other fields.
<code>\$ srun</code>	Runs a job step in real time within an allocation. Used either directly (to request resources and launch a program) or inside a <code>\$ sbatch</code> script to start parallel tasks across allocated nodes.

All SLURM commands have extensive help through their man pages e.g.

```
1 | $ man sbatch
```

Preparing SLURM Submission Script



You can easily create a job script using the [SLURM script generator](#).

Example: Job running on a single node with `compute-normal` partition

The simplest case is that of a job that requires a single node with the following requirements:

- ▶ The job uses 1 node with `compute-normal` partition,
- ▶ The application is a single process,
- ▶ The job will run for 4GiB of RAM and no more than 100 hours,
- ▶ The job is given the name “test123” and
- ▶ The user should be emailed when the job starts and stops or aborts.

Supposing the application is called `myCode` and takes no command line arguments, the following submission script runs the application in a single job

```
1 | #!/bin/bash
```

```

2
3 # Submit this script with:
4 # sbatch <this_script.sub>
5
6 ##### SLURM header #####
7 ##### Auto generated for BistKA | KVIS Mini-HPC Cluster #####
8
9 #SBATCH --job-name="test123"
10 #SBATCH --nodes=1
11 #SBATCH --cpus-per-task=2
12 #SBATCH --partition=compute-normal
13 #SBATCH --account=hpcuser01
14 #SBATCH --mem=4GB
15 #SBATCH --time=100:00:00
16
17 #####
18
19 # Load modules (example)
20 # module load gcc
21 # module load openmpi
22
23 # Your application commands go below
24 ./myCode

```

The script begins with `#!/bin/bash`, known as the shebang. This tells the system to interpret the submission script using the Bash shell, so the file is both a SLURM job script and a standard Bash script.

Next, the script contains several lines starting with `#`. In a Bash script these lines are comments and are ignored by the shell. However, any line that starts with `#SBATCH` is treated by SLURM as a directive that requests specific scheduling resources. It is crucial that all `#SBATCH` lines appear before any executable commands in the script; if a `#SBATCH` directive appears after a Bash command, SLURM will ignore it.

The directive `#SBATCH --nodes=n` specifies how many compute nodes the scheduler should allocate for the job. In the example, only one node is requested. Be careful with single-process, multi-threaded applications (such as MATLAB): they cannot run across multiple nodes, so requesting more than one node (e.g. `#SBATCH --nodes=2`) will usually result in one node doing all the work while the others remain idle.

The maximum runtime (walltime) is set using `#SBATCH --time=T`, where **T** is in the format hours:minutes:seconds (e.g. `01:00:00` for one hour). The job should normally complete within this limit. Once the specified time is reached, SLURM will terminate the job even if it is still running.

You can also assign a human-readable name to the job with `#SBATCH --job-name="name"`,

which makes it easier to identify in `$ squeue` , `$ sacct` , and other SLURM tools.

After the `#SBATCH` section, the remainder of the file is an ordinary Bash script that defines the commands to be executed as part of the job. By default, the job starts in the directory from which it was submitted (unless changed in the script) and inherits the user's environment at submission time (including loaded modules). In the example, this final section simply runs the `myCode` executable as the main workload of the job.

Example: Job running on a single node with `compute-normal` partition and CPU parallel

As a second example, suppose we want to run an MPI application called `myMPICode` with the following requirements:

- ▶ The run uses 1 nodes with 32 CPU-cores,
- ▶ The job will run for 4GiB of RAM and not run for more than 100 hours,
- ▶ The job is given the name "test123" and
- ▶ The user should be emailed when the job starts and stops or aborts.

Supposing no input needs to be specified, the following submission script runs the application in a single job

```

1  #!/bin/bash
2
3  # Submit this script with:
4  # sbatch <this_script.sub>
5
6  ##### SLURM header #####
7  ##### Auto generated for BistKA | KVIS Mini-HPC Cluster #####
8
9  #SBATCH --job-name="test123"
10 #SBATCH --nodes=1
11 #SBATCH --ntasks=32
12 #SBATCH --cpus-per-task=1
13 #SBATCH --partition=compute-normal
14 #SBATCH --account=hpcuser01
15 #SBATCH --mem=4GB
16 #SBATCH --time=100:00:00
17
18 #####
19
20 # Load modules (example)
21 # module load gcc
22 # module load openmpi

```

```

23 |
24 | # Your application commands go below
25 | srun -n 32 ./myMPICode

```

In large part, the script above is similar to the one for a single node job except in this example, `#SBATCH --ntasks=m` is used to reserve `m` cores for the node and to prepare the environment for a MPI parallel run with `m` processes per compute node.

SLURM Partition Checking

SLURM partitions are essentially different queues that point to collections of nodes. On [BistKA](#) there are **9 partitions**:

Partition	Walltime	Priority	Remark
compute-normal	2 Days	1000	
compute-long	4 Days	500	
compute-devel	6 Hrs	1500	Default Partition
gpu4500-normal	2 Days	1000	
gpu4500-long	4 Days	500	
gpu4500-devel	6 Hrs	1500	
gpu4000-normal	2 Days	1000	<i>Currently not available</i>
gpu4000-long	4 Days	500	<i>Currently not available</i>
gpu4000-devel	6 Hrs	1500	<i>Currently not available</i>

- ▶ [compute*](#) : these are the partition that use CPU for execution. The one your jobs will go to [compute-devel](#) partition if you do not specify a partition in your job submission script.
- ▶ [gpu*](#) : these are the partition that contains compute nodes with GPU accelerators, [RTX PRO 4500](#) and [RTX PRO 4000](#) . You only need to specify this partition if your job requires GPU accelerators.

You can specify the SLURM partition by adding the `#SBATCH --partition=` directive to the top of your submission script so adding:

```
1 | #SBATCH --partition=gpu4000-normal
```

will send your job to the RTX PRO 4000 GPU partition. You can see the current state of the partitions with the `$ sinfo` command.

```
1 | $ sinfo
2 | PARTITION      AVAIL  TIMELIMIT  NODES  STATE NODELIST
3 | compute-normal  up 2-00:00:00      3  idle bistka-compute-01,bistka-
4 | gpu-02,bistka-storage-01
5 | compute-long    up 4-00:00:00      2  idle bistka-compute-01,bistka-
6 | gpu-02
7 | compute-devel*  up 6:00:00         1  idle bistka-master
8 | gpu4500-normal  up 2-00:00:00      1  idle bistka-gpu-01
   | gpu4500-long    up 4-00:00:00      1  idle bistka-gpu-01
   | gpu4500-devel   up 6:00:00         1  idle bistka-gpu-01
```

The command `$ sinfo` can output the information in a node-oriented fashion, with the argument `-N -l`

```
1 | $ sinfo -N -l
2 | Sun Nov 09 07:38:28 2025
3 | NODELIST      NODES      PARTITION      STATE CPUS      S:C:T MEMORY
4 | TMP_DISK WEIGHT AVAIL_FE REASON
5 | bistka-compute-01      1 compute-normal  idle 128      2:32:2 512000
6 | 0      1 (null) Node unexpectedly re
7 | bistka-compute-01      1 compute-long   idle 128      2:32:2 512000
8 | 0      1 (null) Node unexpectedly re
9 | bistka-gpu-01          1 compute-normal  idle 32       1:16:2 86016
10 | 0      1 (null) Node unexpectedly re
11 | bistka-gpu-01          1 compute-long   idle 32       1:16:2 86016
   | 0      1 (null) Node unexpectedly re
   | bistka-gpu-02          1 compute-normal  idle 32       1:16:2 86016
   | 0      1 (null) Not responding : Not
   | bistka-gpu-02          1 compute-long   idle 32       1:16:2 86016
   | 0      1 (null) Not responding : Not
   | bistka-master          1 compute-devel*  idle 64       2:16:2 98304
```

```

0      1      (null) none
bistka-storage-01      1 compute-normal      idle 64      2:16:2      86016
0      1      (null) none

```

SLURM Job Submission

After you have prepared a submission script (for example, [script.sh](#) ) , you send the job to the scheduler with:

```
1 | $ sbatch script.sh
```

SLURM will immediately print a numeric **JobID** and return you to the Linux prompt. At this stage, the job has been accepted by the system and placed in the queue.

The job will usually remain in the **PENDING** state until the required resources become available. How long it waits depends on several factors, such as overall cluster load, your requested CPUs/GPUs/memory, and any partition or time-limit constraints.

When the job actually starts running, Slurm creates one or more output files named like **slurm-<JobID>.out** (for example, **slurm-1234.out**). By default, these files are written to the directory where you ran sbatch, unless your script explicitly changes to another directory.

SLURM Interactive Job Submission

On the **BistKA** , interactive jobs are used when you need a live shell on a compute node instead of running heavy commands on the master node. Typical use cases include testing and debugging code, exploring data, trying modules, or quickly checking that a program runs correctly before you turn it into a proper batch job. Instead of typing **\$ srun** directly each time, you should start an interactive session via the helper script **\$ sinteractive** , which automatically sets up a suitable **\$ srun --pty ...** command, chooses a partition (e.g. **compute-devel** by default), and asks for an account if needed. Once the job starts, you are “inside” a Slurm job on a compute node, but you interact with it just like a normal Linux shell.

In practice, you log in to **BistKA** , then launch an interactive job with **\$ sinteractive** and let Slurm allocate resources (CPUs, memory, and optionally GPUs) for you. While you are in that interactive shell, you can load modules, run compilers or Python, and test your code without overloading the login node. When you finish, typing **\$ exit** closes the shell and immediately frees the resources for other users. Interactive jobs should be **short and lightweight**; longer production runs should be submitted with **\$ sbatch** instead.

Example: Interactive GPU Job with Time Limit

```
1 | # Request 1 GPU, 4 CPU cores, 8 GiB RAM for up to 1 hour
2 | $ sinteractive -A project_account -p compute-devel --cpus-per-task=4 --
3 | mem=8G --time=01:00:00
4 |
5 | # Once the shell appears on a compute-devel partition:
6 | $ ml gnu12
7 | $ ml openmpi4
8 | $ ml miniforge3
9 | $ python train_small_model.py
10 |
11 | # Release the resources when finished
    $ exit
```

SLURM Job Array Submission

On the [BistKA](#), a SLURM Job Array is a convenient way to submit many similar jobs with a single `sbatch` command. Instead of writing and submitting 5 separate scripts, you submit one script with an `--array` option, and SLURM launches one job instance (a task in the array) for each index in that array. This is ideal for workflows like parameter sweeps, processing many input files, or running the same program on multiple samples or seeds.

Each task in the array runs the same script, but SLURM gives it a different array index, accessible through the environment variable `SLURM_ARRAY_TASK_ID`. You then use that ID inside the script to choose a specific input file, parameter set, or output name. SLURM treats each array element like a separate job for scheduling, accounting, and resource usage, but managing them is much easier: you can submit, monitor, and cancel the whole group via a single [JobID](#).

Suppose you have input files:

```
1 | input_0.dat
2 | input_1.dat
3 | ...
4 | input_4.dat
```

You can process them with a single job array:

```
1 | #!/bin/bash
2 |
3 | # Submit this script with:
4 | # sbatch <this_script.sub>
```

```

5 | ##### SLURM header #####
6 | ##### Auto generated for BistKA | KVIS Mini-HPC Cluster #####
7 |
8 |
9 | #SBATCH --job-name="array_test"
10 | #SBATCH --nodes=1
11 | #SBATCH --cpus-per-task=2
12 | #SBATCH --partition=compute-devel
13 | #SBATCH --account=hpcuser01
14 | #SBATCH --mem=4GB
15 | #SBATCH --time=1:00:00
16 | #SBATCH --array=0-4
17 |
18 | #####
19 |
20 | module purge
21 | ml miniforge3
22 |
23 | input="input_${SLURM_ARRAY_TASK_ID}.dat"
24 | output="output_${SLURM_ARRAY_TASK_ID}.dat"
25 |
26 | python my_script.py --input "$input" --output "$output"

```

Submit once:

```
1 | sbatch array_job.sh
```

Slurm will create 5 tasks with IDs `0, 1, ..., 4`. Inside each task, `SLURM_ARRAY_TASK_ID` takes a different value, so the script picks the matching input/output pair.

SLURM Job Monitoring

`$ squeue` and `$ myqueue` is the main command for monitoring the state of systems, groups of jobs or individual jobs.

The command `$ squeue` prints the list of current jobs. The list looks something like this:

```

1 | $ squeue
2 | JOBID PARTITION   NAME     USER ST       TIME  NODES NODELIST(REASON)
3 |   2497  compute-d  test1.14  bob  R        0:07      1 bistka-master
4 |   2499  compute-d  test1.35  mary R        0:22      1 bistka-master

```

```
5 | 2511 compute-d ask.for steve PD 0:00 1 (Resources)
```

When you run `squeue` with the default format, you'll typically see columns like:

- ▶ **JOBID** – The unique identifier of the job.
- ▶ **PARTITION** – The partition (queue) where the job was submitted.
- ▶ **NAME** – The job name, usually set in the script with `#SBATCH --job-name=...`
- ▶ **USER** – The owner of the job.
- ▶ **ST** – The job state, shown as a short code, for example: **R** = Running, **PD** = Pending (waiting for resources or other conditions), **CA** = Cancelled, **CF** = Configuring (resources being prepared), **CG** = Completing (finishing up), **CD** = Completed, and **F** = Failed
- ▶ **TIME** – The elapsed run time for the job.
- ▶ **NODES** – The number of nodes allocated (or requested) for the job.
- ▶ **NODELIST(REASON)** – If the job is running, this shows the node list. If it is pending, this field usually shows the reason it is not yet running (e.g. Resources).

Some useful `squeue` options:

- ▶ `$ squeue -u <user>` : Show only the jobs belonging to a specific user (e.g. `$ squeue -u bob`).
- ▶ `$ squeue -l` : Display a more detailed, “long” listing with additional fields.

SLURM Job Deletion

Use the `$ scancel` command to delete a job, e.g. `$ scancel 2479` to delete job with id 2479.

```
1 | scancel 2497
```

You can cancel your own jobs at any time using `$ scancel` , whether they are still **PENDING** in the queue or already **RUNNING** . However, you are not allowed to cancel jobs belonging to other users.

After you issue an `$ scancel` command, there is usually a brief delay before the job actually disappears from the queue and any running processes are terminated, as SLURM needs a moment to propagate the cancellation across the cluster.

SLURM Environment Variables

When a job begins running, Slurm automatically sets a number of environment variables that you can use inside your job script to control behavior and file locations. Some of the most commonly used variables are:

- ▶ **SLURM_SUBMIT_DIR** : the directory from which you ran the sbatch command.
- ▶ **SLURM_JOB_NODELIST** : the list of compute nodes assigned to the job.
- ▶ **SLURM_JOB_ID** : the unique numeric ID that SLURM assigns to the job.

SLURM_JOB_ID is useful to tag job specific files and directories, typically output files or run directories. For instance, the submission script line

```
1 | myApp > $SLURM_JOB_ID.out
```

runs the application **myApp** and redirects the standard output to a file whose name is given by the job ID. The job ID is a number assigned by SLURM and differs from the character string name given to the job in the submission script by the user.

On **BistKA**, jobs normally start in the directory where **\$ sbatch** was executed, so you do not usually need to reference **SLURM_SUBMIT_DIR** just to run your code. This is different from some other schedulers (e.g. Torque), which may start jobs in the user's home directory instead.

However, **SLURM_SUBMIT_DIR** is still very useful when you need to copy data to or from a specific directory, especially if your script changes directories or runs from a different path. In such cases, using **SLURM_SUBMIT_DIR** ensures that your script always knows where the original submission directory is located.

For other variables, please visit <https://slurm.schedmd.com/sbatch.html> 

Job Credit

Job Credit

On the [BistKA](#), **each SLURM account is given a pool of Service Hours (SHr) as computing credit**, often with a fixed expiry date for project and classe accounts. The command-line tool `mycredit` provides a simple way for users to check:

- ▶ How much credit has been allocated to each account,
- ▶ How much has been used so far,
- ▶ How much is remaining, and
- ▶ When that credit will expire.

`$ mycredit` reads this information from the SLURM accounting system and prints it in a human-friendly summary table. This helps you plan your jobs, avoid unexpected **“out of credit”** problems, and track usage within project or class accounts.

By default, `$ mycredit` reports account-level usage in Service Hours (SHr), but it can also show a per-user breakdown inside each account.

Default Job Credit

Credit	SHr	Detail/Policy
User Log-in Account	500 SHr	Personal use in classes (for students) or small, related project work
Student Project / Teacher Project	10,000 SHr	Research projects for students and teachers
School Class	2,500 SHr	Computational courses and class-related assignments or activities



If your project or school class requires additional job credits, **you must submit an Additional Project Resources Request Form (FM-HPC-05)**.



The job credit quota for the user-login account are not granted top-up.

Basic Usage

```

1 $ mycredit
2
3
4 Account      Allocation(SHr)      Service Hour
5 Remaining(%) Expiry Date      Used(SHr)      Remaining(SHr)
6 -----
7 - -----
25sp01          10000.00          6.42          9870.58
98.71% 2026-02-28
hpcuser01        500.00          3.78          496.22
99.24% -

```

Running `$ mycredit` with no options prints a table summarizing all SLURM accounts you belong to. The default columns are:

- ▶ **Account** – SLURM account / project name
- ▶ **Allocation(SHr)** – total Service Hours allocated to that account
- ▶ **Used(SHr)** – total Service Hours already consumed
- ▶ **Remaining(SHr)** – Service Hours still available
- ▶ **Remaining(%)** – remaining credit as a percentage of the allocation
- ▶ **Expiry Date** – date when the account's credit expires

Per-user Detail Inside Each Account

If you want to see how credit is used by individual users inside each account:

```

1 $ mycredit -d
2
3 Account      User      Used(SHr)      Used(%)
4 -----
5 25sp01      hpcuser01      5.70      88.60%
6             hpcuser02      0.73      11.40%

```

7		Total	6.43	100.00%
8		-----	-----	-----
9	hpcuser01	hpcuser01	3.80	100.00%
10		Total	3.80	100.00%
11		-----	-----	-----

This switches to a per-user view and shows, for each account:

- ▶ **Account** – SLURM account name
- ▶ **User** – user log-in account
- ▶ **Used(SHr)** – Service Hours used by that user
- ▶ **Used(%)** – that user’s share of the account’s total usage

Selecting Columns (Advanced)

Advanced users can choose exactly which fields to display using `-o` or `--format` with a comma-separated list of field names:

```

1 $ mycredit -o account,allocation,used,remaining_percent,expiry
2
3           Service Hour
4 Account      Allocation(SHr)      Used(SHr) Remaining(%) Expiry Date
5 -----
6 25sp01        50000.00             6.42      98.71% 2026-02-28
7 hpcuser01      500.00             3.78      99.24% -

```

Common field names include:

- ▶ **account** – SLURM account name
- ▶ **allocation** – Allocation(SHr)
- ▶ **used** – Used(SHr)
- ▶ **remaining** – Remaining(SHr)
- ▶ **used_percent** – Used(%)
- ▶ **remaining_percent** – Remaining(%)
- ▶ **expiry** – Expiry Date
- ▶ **allocation_system** – Allocation(SU), system-level Service Units
- ▶ **used_system** – Used(SU)

► **remaining_system** – Remaining(SU)

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

Disk Quota

Disk Quota

On the [BistKA](#), user data is stored on a **Lustre filesystem** with quotas applied to both your Home directory and any Project directories you belong to. The `$ myquota` command is a small, read-only helper that queries Lustre and prints a concise summary of your disk usage and limits in one place. It shows both **block quotas** (how much space in GiB/TiB you're using vs. your limit) and **inode quotas** (how many files you've created vs. your limit) for your personal Home folder and all visible project folders.

Internally, `$ myquota` reads Lustre quotas for your **user ID** and for each **project ID** associated with project directories under **/lustrefs/project**. It only lists project folders that you can actually see and that have a valid Lustre project ID; anything else is quietly skipped.

Default Disk Quota

System	Size	Path	Detail/Policy
User Home	50 GiB	/lustrefs/home or \$HOME	Personal storage space for each user. Intended for code, configuration files, and small-medium data.
Project Home	300 GiB	/lustrefs/project	Shared storage for Student Project, Teacher Project, and School Class. Accessible to all members of a group.



If your project requires additional disk space, **you must submit an Additional Project Resources Request Form (FM-HPC-05)**.



The disk quota for the user-login account are not granted extension.

Basic Usage

Most users only need this command:

```

1 $ myquota
2
3           Block Quota Limits           Inode Quota
4 Limits
5           Used           Size           IUsed
6 Inodes
7 -----
--
/lustrefs/home/hpcuser01      771.8M      50G      24841
1100000
/lustrefs/project/25sp01      25k        300G        1
11000000

```

Running `$ myquota` :

- ▶ Detects your user-login account.
- ▶ Locates your Home directory (usually `/lustrefs/home/$USER`).
- ▶ Scans all project folders under `/lustrefs/project`.
- ▶ Prints a formatted table with one row per path (Home and each visible Project).

Where:

- ▶ **Used / Size (Block Quota)** show space usage vs. quota.
- ▶ **IUsed / Inodes (Inode Quota)** show how many files you have vs. the file-count limit.

If you have no visible projects (or no project IDs are defined yet), `$ myquota` will still show your Home directory and a placeholder row for the project area.

When to Use

Use `$ myquota` whenever you want to:

- ▶ Check **how close you are to your storage** limit in Home or project folders.
- ▶ Diagnose “disk quota exceeded” errors before or after running jobs.
- ▶ Confirm that a new project directory is correctly attached to a Lustre project ID and visible under your account.

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

Login via Visual Studio Code

Login via Visual Studio Code



BistKA blocks all `vscode-server` processes started by **Remote - SSH** extension, as it consumes a large amount of system resources when working with the cluster.

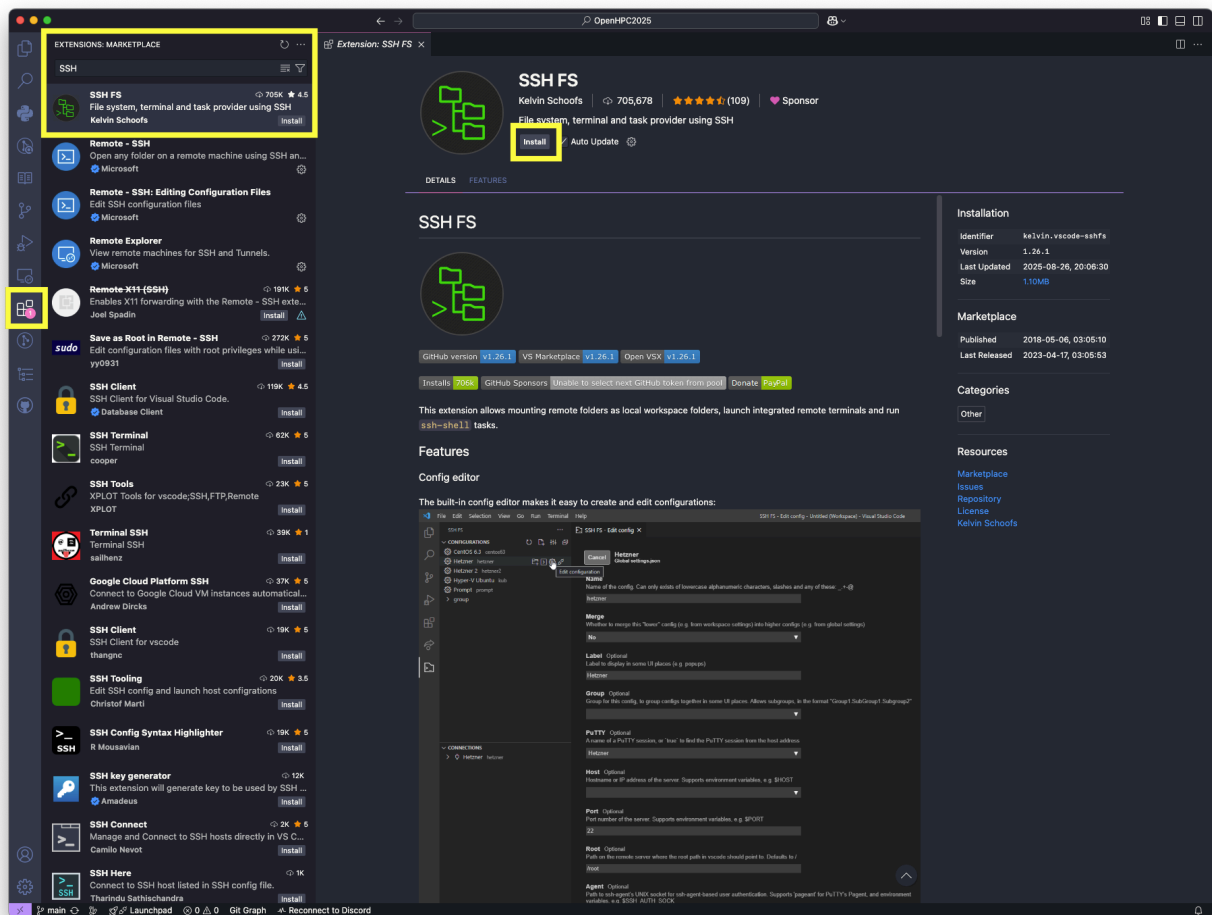
Visual Studio Code (VS Code) is a free, cross-platform source-code editor that can connect to BistKA over SSH via [SSH FS](#) package and let you work on files directly on the cluster while using a familiar graphical editor on your own computer.

With the [SSH FS](#) extension, VS Code opens a remote workspace on the **BistKA** master node so you can:

- ▶ Browse and edit project files stored under your BistKA home directory
- ▶ Use the built-in terminal to run basic commands, manage modules, and submit SLURM jobs

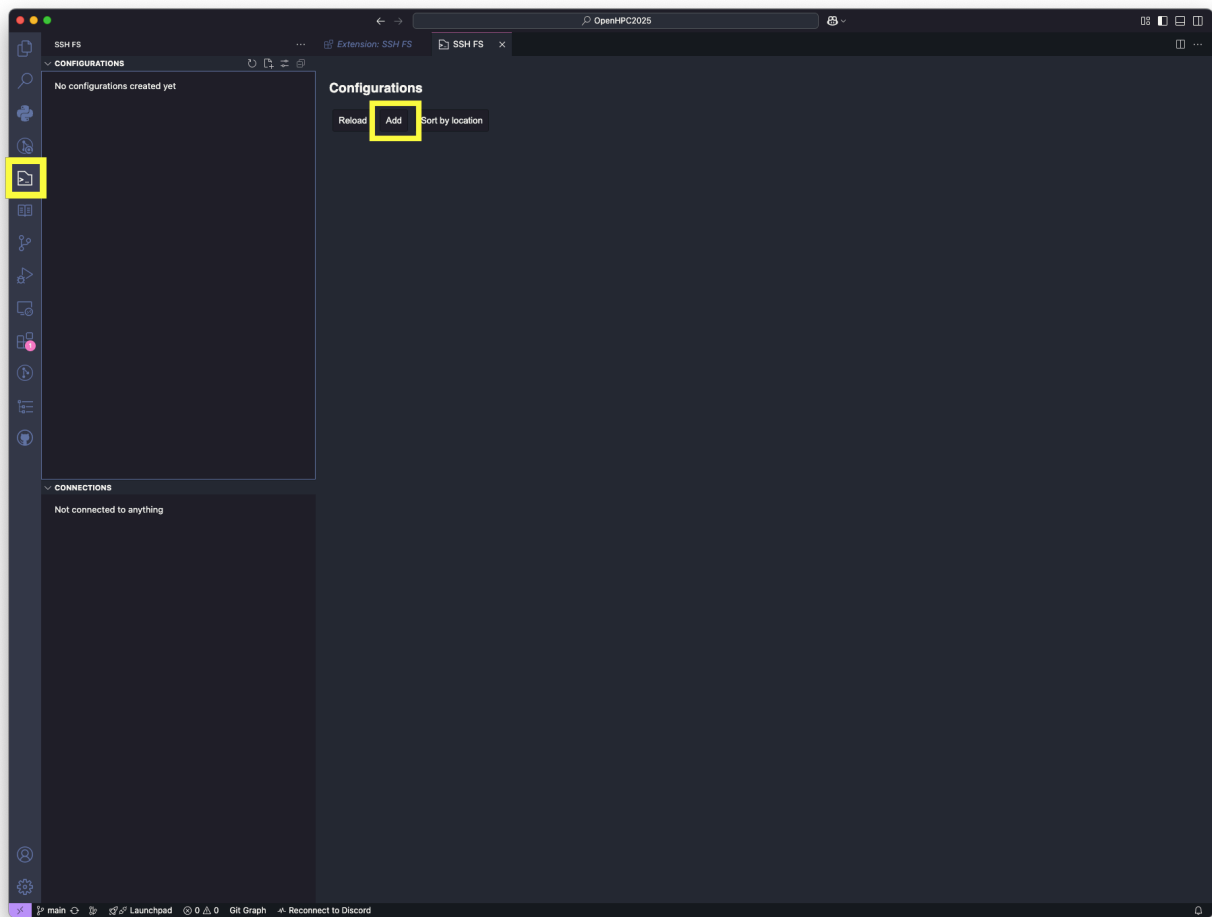
Install the SSH FS extension

1. Open VS Code.
2. Click the [Extensions](#) icon on the left sidebar.
3. In the search box, type [SSH FS](#) .
4. Find [SSH FS](#) and click [Install](#) .

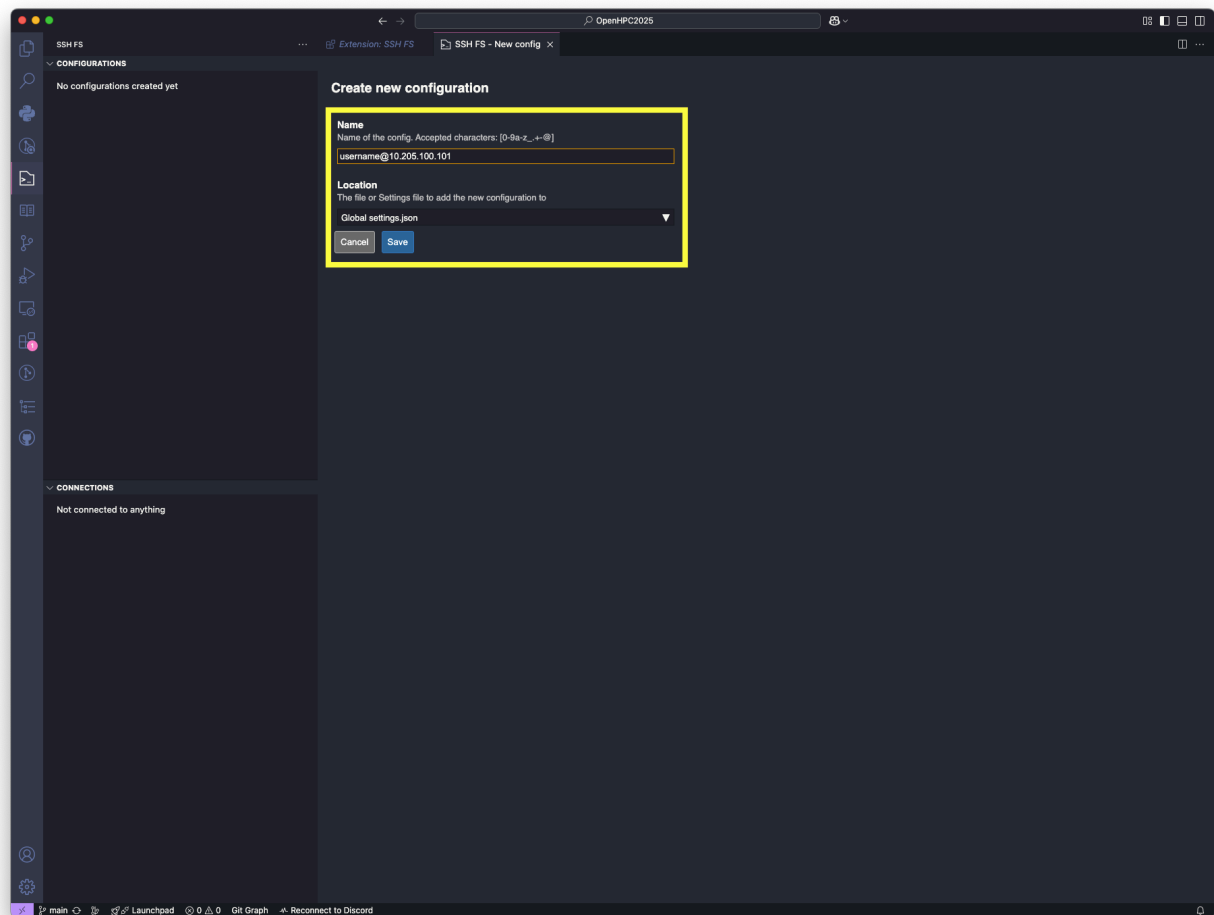


Add A New SSH FS Connection

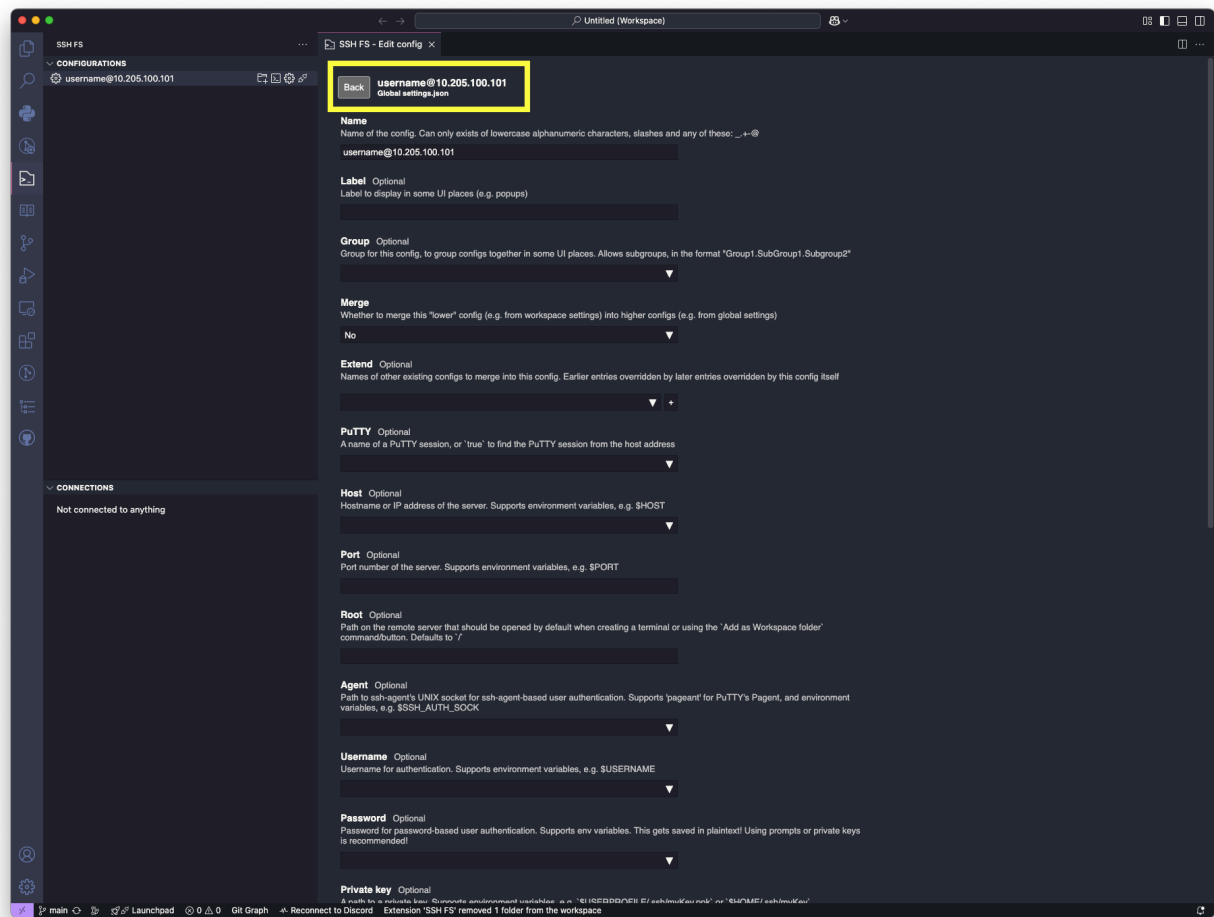
1. Click the **SSH FS** icon on the left sidebar and the click the **Add** button.



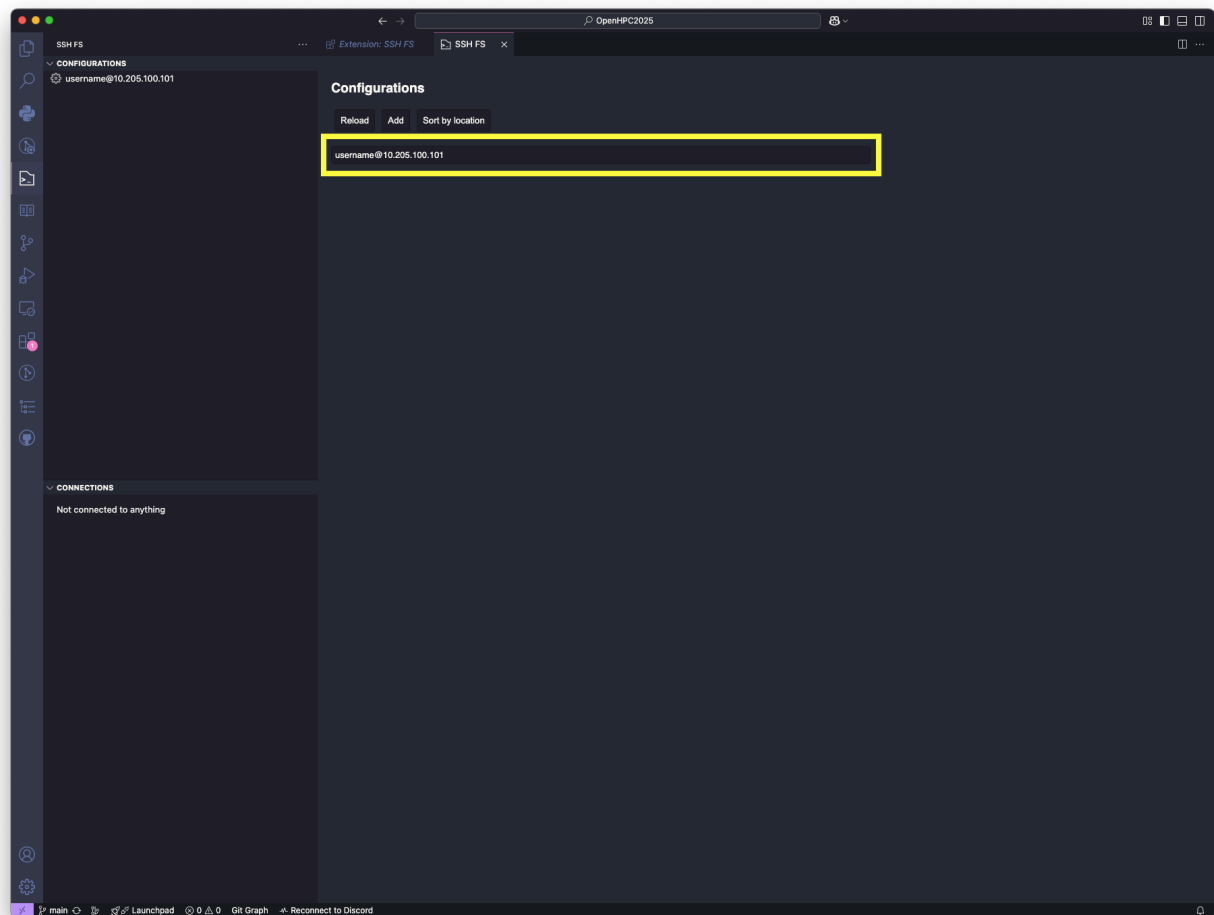
2. In the **Name** field, enter `username@10.205.100.101` , where **username** is your BistKA user log-in account.



3. Click the **Back** button to return to the **SSH FS** list.



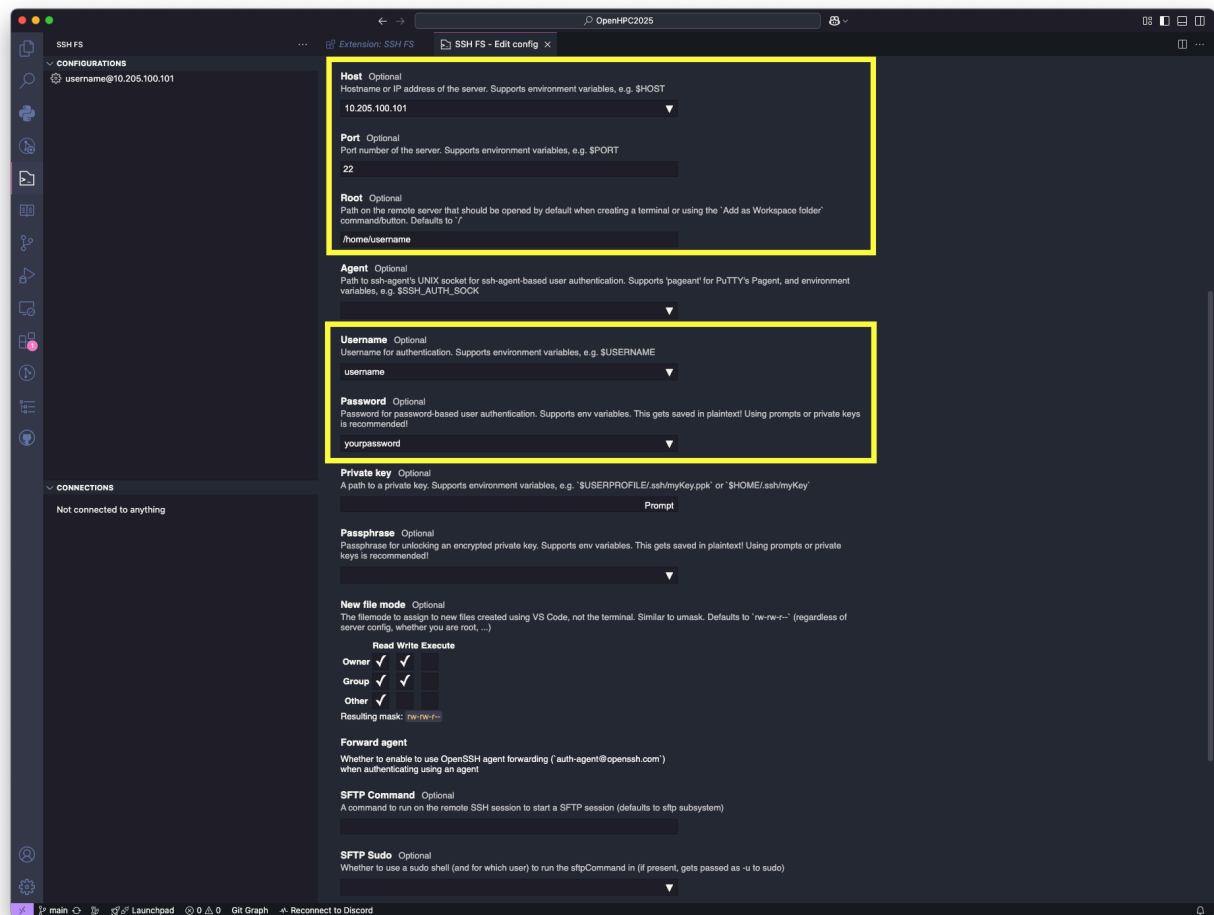
4. Click on your new entry `username@10.205.100.101` to open the connection settings.



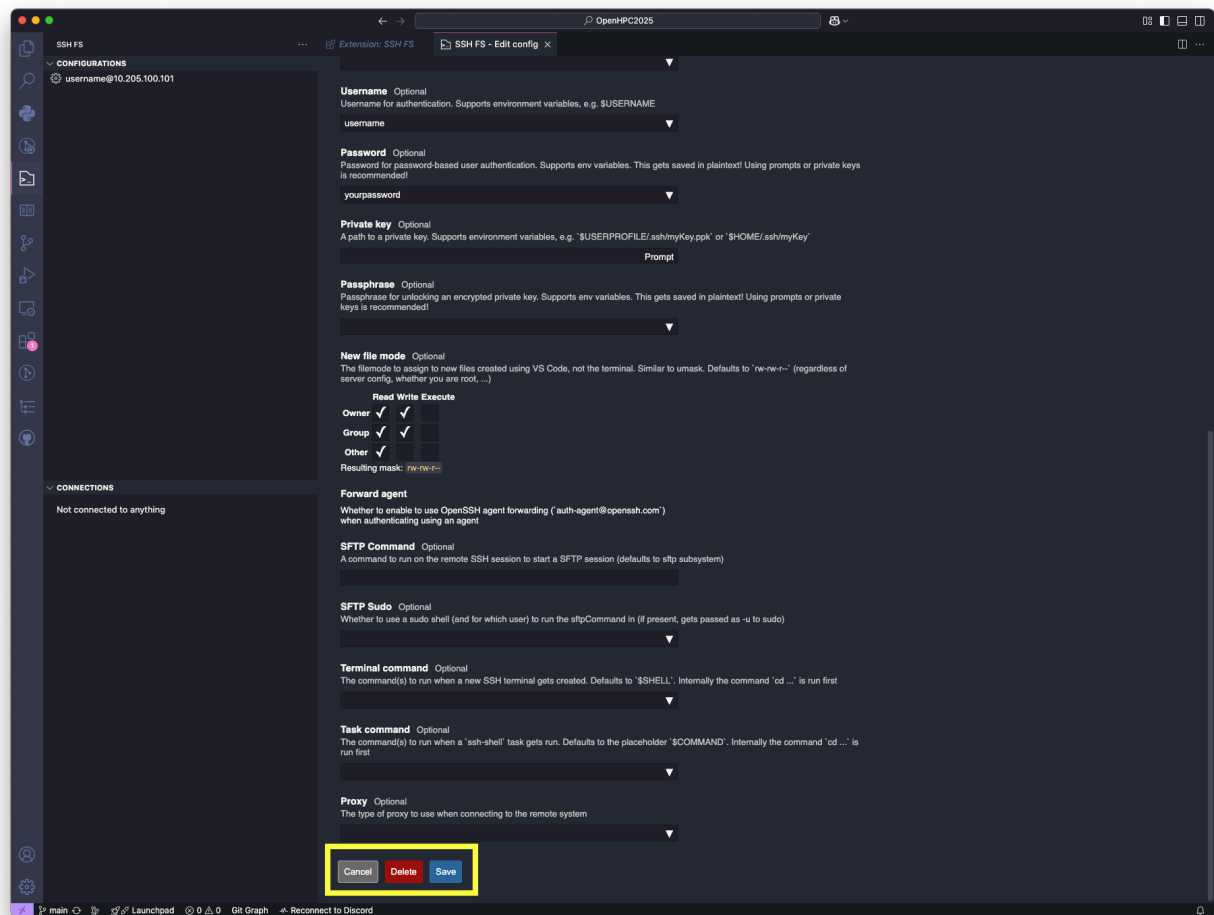
Configure Connection Settings

Fill in the fields as follows:

- ▶ **Host:** 10.205.100.201
- ▶ **Port:** 22
- ▶ **Root:** /lustrefs/username (replace **username** with your BistKA user lo-gin account)
- ▶ **Username:** your BistKA user lo-gin account
- ▶ **Password:** your BistKA password



Then click **Save** at the bottom of the page.



How to Use the Connection

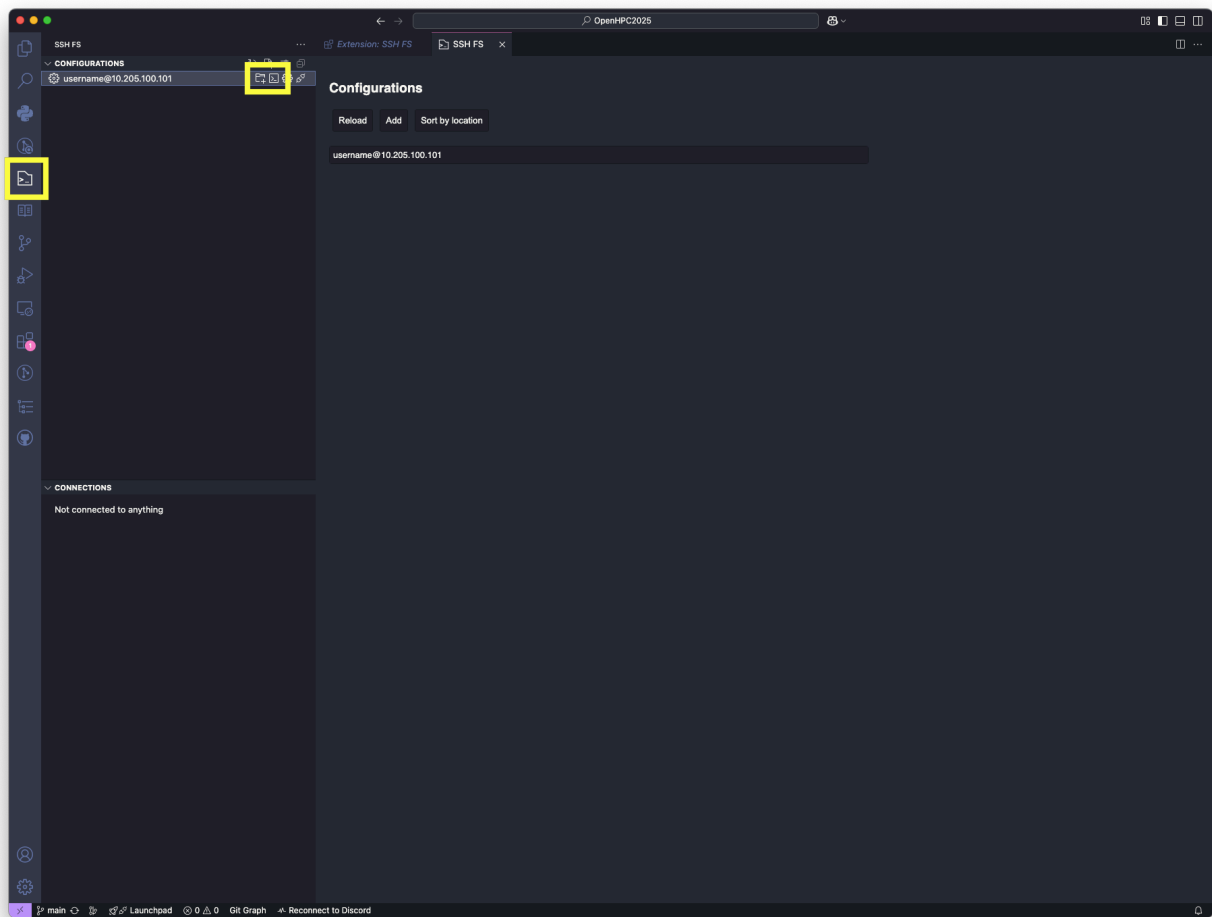
You can use the connection in two ways:

1. Workspace Folder

- At the **SSH FS** configuration, click **Add as Workspace Folder**
- This mounts **/lustrefs/username** into your VS Code workspace so you can browse and edit files.

2. SSH Terminal

- At the **SSH FS** configuration, click **Open Remote SSH Terminal**
- This opens a terminal session to **BistKA** inside VS Code, which you can use to run commands and submit **SLURM** jobs.



© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

Conda and Pip via Miniforge3

Conda and Pip via Miniforge3

pip is Python's standard package installer—it pulls libraries from the Python Package Index (PyPI) and installs them into your current interpreter or virtual environment. **conda** is a cross-platform environment and package manager that creates isolated environments and installs prebuilt binaries for both Python and non-Python dependencies (e.g., CUDA, MKL), which is especially helpful on HPC systems.

A common workflow is: create a fresh **conda** environment, install as much as possible from the conda-forge channel, then use **pip** inside that activated environment for anything not available on **conda**. This keeps dependencies reproducible and avoids conflicts; avoid installing into base or using `pip --user`. On **BistKA**, load **Miniforge3/25.3.0-3** to get conda/mamba and use them within a SLURM job.

How they differ

- ▶ **Scope:** **pip** handles Python packages; **conda** handles Python and non-Python deps (compilers, MKL, CUDA libs, etc.).
- ▶ **Binaries:** **pip** installs wheels/source from PyPI; **conda** installs curated binary builds from channels.
- ▶ **Environments:** **pip** relies on venv/virtualenv; **conda** creates and manages its own envs.
- ▶ **Dependency solving:** **conda** solves the whole stack (often fewer compiler headaches). **pip** resolves Python deps only.

When to use which

- ▶ **Try \$ conda first** for scientific/ML stacks (NumPy/SciPy/PyTorch/CUDA), or when you need native libraries.
- ▶ Use **\$ pip** for packages that aren't on conda, or to add a few extras inside an already-activated conda env.
- ▶ It's common to **mix**: install the heavy stack with **\$ conda**, then **\$ pip install** the missing Python-only libs.

Using conda & pip on BistKA (via module load Miniforge3/25.3.0-3)

1) Load Miniforge (conda/mamba)

```
1 | $ module load Miniforge3/25.3.0-3
2 |
3 | # If conda is not initialized by the module on your shell:
4 | # source "$EBROOTMINIFORGE3/etc/profile.d/conda.sh"
5 | # EBROOTMINIFORGE3 is set by Lmod
```

2) Create a clean environment (recommended)

Never install into base. Create a per-project env under `$HOME` or project folder.

Option A: named env

```
1 | $ mamba create -n myenv python=3.12 -y
2 | # or: $ conda create -n myenv python=3.12 -y
3 |
4 | $ conda activate myenv
```

Option B: path-based env (explicit path)

```
1 | $ conda create -p /lustrefs/home/$USER/conda/envs/myenv python=3.12 -y
2 | $ conda activate /lustrefs/home/$USER/conda/envs/myenv
```



Tip: Use project storage if appropriate:
`/lustrefs/project/<project_folder>/envs/myenv`

3) Install packages

Prefer `$ conda` / `$ mamba` **when available**

```
1 | $ mamba install numpy pandas scipy -y
2 | # or: $ conda install ...
```

Use `$ pip` for what's missing from conda-forge

Run `$ pip` inside the activated env (no `--user`).

```
1 | $ python -m pip install --upgrade pip
2 | $ pip install polars fastapi # example
3 |
4 | # (Optional) reduce cache size:
5 | # PIP_CACHE_DIR=/lustrefs/home/$USER/.cache/pip pip install <pkg>
```

4) Save / reproduce your environment

```
1 | # Conda-style
2 | $ conda env export --from-history > environment.yml
3 | $ mamba env create -n myenv2 -f environment.yml
4 |
5 | # Pip-style (for pip-installed extras)
6 | $ pip freeze > requirements.txt
7 | $ pip install -r requirements.txt
```

5) Remove the environment

By name:

```
1 | $ conda env remove -n myenv
2 | # (or) $ mamba env remove -n myenv
```

By path (if you created it with -p):

```
1 | $ conda env remove -p /lustrefs/home/$USER/conda/envs/myenv
```

Clean caches to free space

```
1 | $ conda clean --all --yes
2 | $ pip cache purge    # only if you used pip
```

If any leftover directory remains

```
1 | $ rm -rf /lustrefs/home/$USER/conda/envs/myenv
```

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

JupyterLab via Miniforge3

JupyterLab via Miniforge3

Jupyter (Notebook/Lab) lets you run Python code interactively, visualize results, and keep code + outputs together in a single document. On [BistKA](#), Jupyter runs on compute nodes via [SLURM](#) and is viewed from your laptop through SSH port-forwarding.

Typical uses: exploratory data analysis, plotting, small-to-medium experiments, log inspection, and prototyping job scripts.

Load Miniforge and create/activate an environment

```
1 | $ module load Miniforge3/25.3.0-3
2 |
3 | # one-time (per project)
4 | $ mamba create -n mynb python=3.12 -y
5 | $ conda activate mynb
6 |
7 | # Jupyter + optional kernel registration
8 | $ mamba install -y jupyterlab ipykernel
9 | $ mamba install -y pandas matplotlib numpy scipy seaborn
10 |
11 | # Deactivate conda enviroment after installation
12 | $ conda deactivate
```

Submit job to launch JupyterLab on the compute node

1. Create [SLURM](#) script for running JupyterLab

```
1 | #!/bin/bash
2 |
3 | # Submit this script with:
```

```
4 # sbatch <this_script.sub>
5
6 ##### SLURM header #####
7 ##### Auto generated for BistKA | KVIS Mini-HPC Cluster #####
8
9 #SBATCH --job-name="jupyter"
10 #SBATCH --nodes=1
11 #SBATCH --cpus-per-task=4
12 #SBATCH --partition=compute-normal
13 #SBATCH --account=hpcuser01
14 #SBATCH --mem=4GB
15 #SBATCH --time=12:00:00
16
17 #####
18
19 # Load modules
20 module load Miniforge3/25.3.0-3
21 conda activate mynb
22
23 PORT=$(shuf -i 6000-9999 -n 1)
24 USER=$(whoami)
25 NODE=$(hostname -s)
26
27 echo -e "
28     JupyterLab is running on: $NODE
29     Job starts at: $(date)
30     Copy/Paste the following command into your local terminal
31     -----
32     ssh -L $PORT:$NODE:$PORT $USER@10.205.100.101
33     -----
34     Open a browser on your local machine with the following address
35     -----
36     http://localhost:${PORT}/?token=XXXXXXXX (see your token below)
37     -----
38     "
39
40 # start a cluster instance and launch jupyter server
41 unset XDG_RUNTIME_DIR
42 if [ "$SLURM_JOBTMP" != "" ]; then
43     export XDG_RUNTIME_DIR=$SLURM_JOBTMP
44 fi
45
46 jupyter lab --no-browser --ip="$NODE" --port="$PORT" --notebook-
47 dir="$(pwd)"
```

2. Submit your job and read your slurm-xxxxx.out

```
1 | $ sbatch jupyter.sub
```

```
1 $ cat slurm-115.out
```

```
3 Jupyter Lab is running on: bistka-gpu-02
```

```
4 Job starts at: Mon Nov 17 06:44:51 AM +07 2025
```

```
5 Copy/Paste the following command into your local terminal
```

```
7 ssh -L 8940:bistka-gpu-02:8940 hpcuser01@10.205.100.101
```

```
9 Open a browser on your local machine with the following address
```

```
11 http://localhost:8940/?token=XXXXXXXX (see your token below)
```

```
14 [I 2025-11-17 06:44:55.678 ServerApp] jupyter_lsp | extension was  
15 successfully linked.
```

```
16 [I 2025-11-17 06:44:55.681 ServerApp] jupyter_server_terminals |  
17 extension was successfully linked.
```

```
18 [I 2025-11-17 06:44:55.685 ServerApp] jupyterlab | extension was  
19 successfully linked.
```

```
20 [I 2025-11-17 06:44:55.691 ServerApp] Writing Jupyter server cookie  
21 secret to
```

```
22 /lustrefs/home/hpcuser01/.local/share/jupyter/runtime/jupyter_cookie_secr  
23 et
```

```
24 [I 2025-11-17 06:44:57.121 ServerApp] notebook_shim | extension was  
25 successfully linked.
```

```
26 [I 2025-11-17 06:44:57.215 ServerApp] notebook_shim | extension was  
27 successfully loaded.
```

```
28 [I 2025-11-17 06:44:57.217 ServerApp] jupyter_lsp | extension was  
29 successfully loaded.
```

```
30 [I 2025-11-17 06:44:57.218 ServerApp] jupyter_server_terminals |  
31 extension was successfully loaded.
```

```
32 [I 2025-11-17 06:44:57.227 LabApp] JupyterLab extension loaded from  
33 /lustrefs/home/hpcuser01/.conda/envs/mynb/lib/python3.12/site-  
34 packages/jupyterlab
```

```
35 [I 2025-11-17 06:44:57.227 LabApp] JupyterLab application directory is  
36 /lustrefs/home/hpcuser01/.conda/envs/mynb/share/jupyter/lab
```

```
37 [I 2025-11-17 06:44:57.229 LabApp] Extension Manager is 'pypi'.
```

```
38 [I 2025-11-17 06:44:57.457 ServerApp] jupyterlab | extension was  
39 successfully loaded.
```

```

40 [I 2025-11-17 06:44:57.457 ServerApp] Serving notebooks from local
41 directory: /lustre/fs/home/hpcuser01
[I 2025-11-17 06:44:57.457 ServerApp] Jupyter Server 2.17.0 is running
at:
[I 2025-11-17 06:44:57.457 ServerApp] http://bistka-gpu-02:8940/lab?
token=523b5e23421e73e02b970db30bcde0ebbaef970c3f8d8508
[I 2025-11-17 06:44:57.457 ServerApp] http://127.0.0.1:8940/lab?
token=523b5e23421e73e02b970db30bcde0ebbaef970c3f8d8508
[I 2025-11-17 06:44:57.457 ServerApp] Use Control-C to stop this server
and shut down all kernels (twice to skip confirmation).
[C 2025-11-17 06:44:57.467 ServerApp]

```

To access the server, open this file in a browser:

```
file:///lustre/fs/home/hpcuser01/.local/share/jupyter/runtime/jpserver-
336507-open.html
```

Or copy and paste one of these URLs:

```

http://bistka-gpu-02:8940/lab?
token=523b5e23421e73e02b970db30bcde0ebbaef970c3f8d8508
http://127.0.0.1:8940/lab?
token=523b5e23421e73e02b970db30bcde0ebbaef970c3f8d8508
[I 2025-11-17 06:44:57.601 ServerApp] Skipped non-installed server(s):
basedpyright, bash-language-server, dockerfile-language-server-nodejs,
javascript-type
script-langserver, jedi-language-server, julia-language-server, pyrefly,
pyright, python-language-server, python-lsp-server, r-languageserver,
sql-language-s
erver, texlab, typescript-language-server, unified-language-server,
vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-
json-languageserver
-bin, yaml-language-server

```

From your laptop: SSH port-forward

1. From the slurm-xxxxx.out, find “**ssh -L XXXX:...**” and copy it

```

1 ...
2 -----
3 ssh -L 8940:bistka-gpu-02:8940 hpcuser01@10.205.100.101
4 -----
5 ...

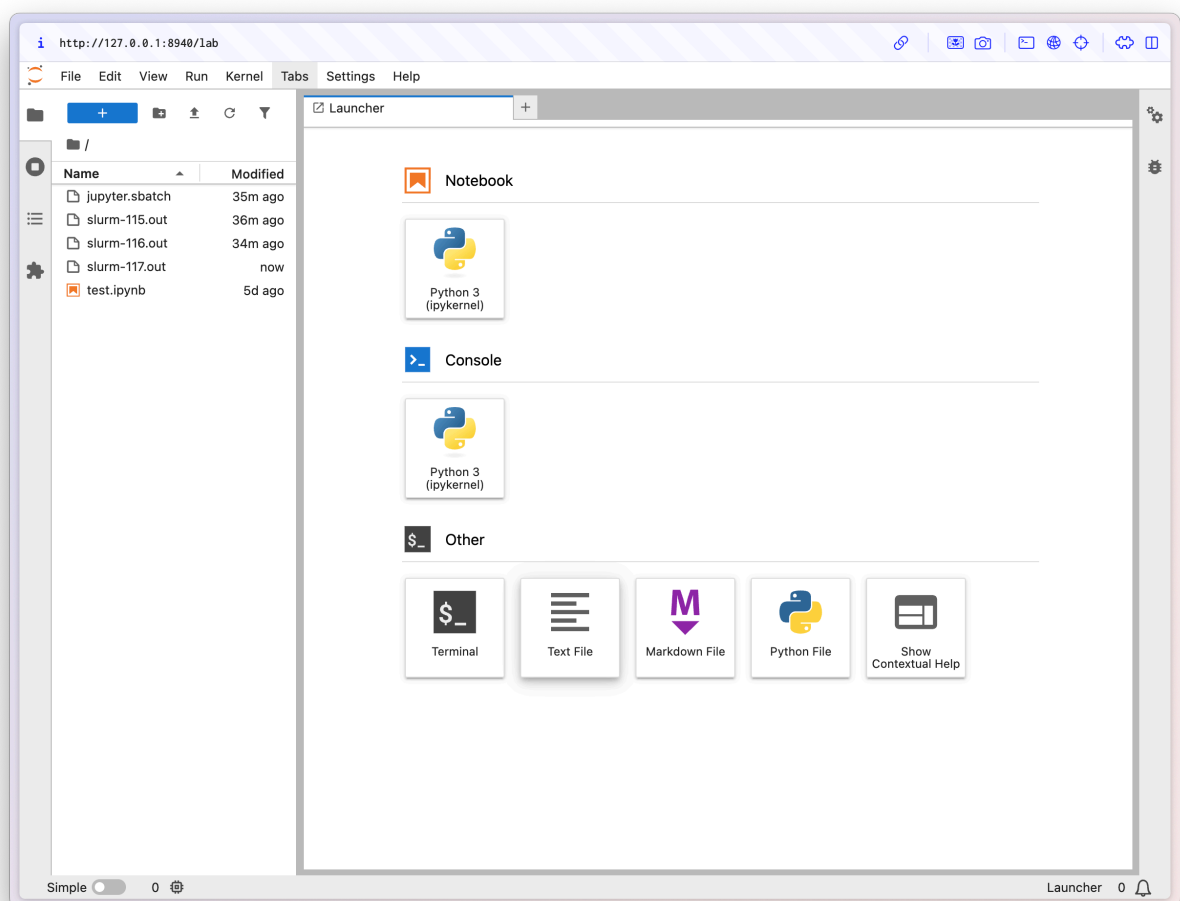
```


2. Open a new terminal on **your laptop** (Tabby / iTerm2 / MobaXterm) and paste the copied command for ssh tunneling to the BistKA:

```
1 | ssh -L 8940:bistka-gpu-02:8940 hpcuser01@10.205.100.101
```

3. Open a browser on **your laptop** with the following address from slurm-xxxxx.out (**please find the line:** [http://127.0.0.1:8940/lab?token=](http://127.0.0.1:8940/lab?token=...)  ...)

```
1 | ...  
2 | http://127.0.0.1:8940/lab?  
3 | token=523b5e23421e73e02b970db30bcde0ebbaef970c3f8d8508  
   | ...
```



Stop the session

When you're done with JupyterLab, close your browser and the local terminal, then cancel the SLURM job with `scancel <jobid>` command.

```
1 | $ scancel <jobid>
```

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

First Job Submission

First Job Submission

This example shows how to submit a very simple test job on the [BistKA](#) using `$ sbatch`. The job will just print some basic information (date, hostname, and current working directory) and then exit.

Example: First Job Submission with `$ sbatch`

1. Create a SLURM Job Script

Create a file called `first_job.sub`:

```
1  #!/bin/bash
2
3  # Submit this script with:
4  # sbatch <this_script.sub>
5
6  ##### SLURM header #####
7  ##### Auto generated for BistKA | KVIS Mini-HPC Cluster #####
8
9  #SBATCH --job-name="first_job"      # Job name
10 #SBATCH --output="slurm-%j.out"     # Standard output file (%j = JobID)
11 #SBATCH --nodes=1                  # Number of node
12 #SBATCH --cpus-per-task=2          # Number of CPU cores
13 #SBATCH --partition=compute-devel  # Partition (queue)
14 #SBATCH --account=hpcuser01        # SLURM account
15 #SBATCH --mem=1GB                  # Memory per job
16 #SBATCH --time=00:05:00            # Max walltime (hh:mm:ss)
17
18 #####
19
20 echo "=== My first job on BistKA ==="
21 echo "Date       : $(date)"
22 echo "Hostname   : $(hostname)"
23 echo "Work dir    : $(pwd)"
```

```
24 | echo "User      : $USER"
25 |
26 | echo
27 | echo "Running a small test command..."
28 | sleep 10
29 | echo "Done."
```

Save the file and make it executable (optional but recommended):

```
1 | $ chmod +x first_job.sub
```

2. Submit the Job

From the directory containing `first_job.sub` , run:

```
1 | $ sbatch first_job.sub
```

You should immediately see something like:

```
1 | Submitted batch job 12345
```

Here **12345** is your `JobID` .

3. Check Job Status

To see whether your job is pending or running:

```
1 | $ squeue -u $USER
2 |
3 |           JOBID PARTITION     NAME     USER ST       TIME  NODES
4 | NODELIST(REASON)
      80 compute-d first_jo hpcuser01  R       0:07      1
      bistka-master
```

When the job finishes, it will disappear from squeue.

4. Inspect the Output

After the job completes, Slurm writes the output to the file specified in `#SBATCH --output`. In this example:

```
1 | $ ls
2 | slurm-12345.out  first_job.sub
```

View the output:

```
1 | cat slurm-12345.out
```

You should see something like:

```
1 | === My first job on BistKA ===
2 | Date       : Mon Nov 10 15:30:12 +07 2025
3 | Hostname   : bistka-compute-01
4 | Work dir   : /lustrefs/home/youruser
5 | User       : youruser
6 |
7 | Running a small test command...
8 | Done.
```

If you see this, your first job submission via `$ sbatch` on **BistKA** is successful.

BistKA HelpDesk

BistKA HelpDesk



To submit a support service ticket, please send an email from an address *under the @kvis.ac.th* domain to hpc@kvis.ac.th using the template below.

New Email Template:

Email Subject:



HelpDesk: <your help or assist needed>

(Please start with "HelpDesk:" and then subject to help or assist)

Email Detail:



Full name:

Student ID:

Project ID (if any):

Problem:

Details (Software, Commands):

Job submission script (if any):

Attachments (if any): (Screenshot, Log files)

Once our team receives your message, the HelpDesk system will automatically send a reply email with the Ticket ID **<TS-ID-XXXXXX>**. When replying, please keep the **Ticket ID in the email subject**

line so we can track your case efficiently.

Guideline:

To help us support you as effectively as possible, please follow these general guidelines:

- ▶ Describe the issue/problem/need clearly and explain how we can help.
- ▶ Provide enough information for us to reproduce and diagnose the problem, such as job scripts, log files, screenshots, software name and version, loaded modules, Python package list, Python/conda/mamba environment details, bash environment, etc.
- ▶ If you are requesting support for software installation, please include links to the official website, Git repository, and/or installation guide.



Each software package has its own licensing scheme. For example, some licenses restrict the physical location or machines where the software may be installed and used, some lock usage to a specific MAC address, and some require a license server to manage access. **We recommend that you carefully review the software's license agreement and ensure that your usage complies with the permitted terms.**

SLURM Script Generator

SLURM Script Generator @BistKA | KVIS Mini-HPC Cluster

Job name:	my job name		
Account (--account):	project or class		
Output file (--output):	slurm-%j.sh		
Partition:	☒ compute-devel ☐ compute-normal ☐ compute-long ☐ gpu4500-devel ☐ gpu4500-normal ☐ gpu4500-long		
Nodes (--nodes):	1 		
MPI:	☐ Use MPI (srun/mpirun)		
Total MPI tasks (--ntasks):	1 		
CPUs per task (--cpus-per-task):	1 		
GPUs per node (--gres=gpu:pro4500:<N>):	0 		
Memory per node (--mem):	4 	GB	
Walltime (--time):	1 	hours	0  minutes
Working directory (--chdir):	\$HOME		

Please fill in Account (--account); it is required for submission.

Generated batch script (sbatch <script.sh>):

COPY


```
#!/bin/bash

# Submit this script with:
# sbatch <this_script.sub>

##### SLURM header #####
##### Auto generated for BistKA | KVIS Mini-HPC Cluster #####

#SBATCH --nodes=1
#SBATCH --cpus-per-task=1
```

Generated one-line submission (sbatch --wrap):

COPY

```
sbatch --nodes=1 --cpus-per-task=1 --partition=compute-devel --mem=4GB --time=1:00:00
```

BistKA | KVIS Mini-HPC SLURM Script Generator

Modified By: Bundit Boonyarit, Ph.D. Candidate, School of Information Science and Technology,
Vidyasirimedhi Institute of Science and Technology (VISTEC)

The latest version of the *Brigham Young University Job Script Generator* is available on [GitHub](#)



Job Credit Calculator

Job Credit (SHr) Usage Calculator @BistKA | KVIS Mini-HPC Cluster

Number of nodes (N_nodes):	1
CPU cores per node (N_cores):	16
Memory per node (Mem, GiB):	64
RTX PRO 4500 GPUs per node (N_4500gpu):	0
RTX PRO 4000 GPUs per node (N_4000gpu):	0
Execution time (T_exec, hours):	1.0
Total credit usage (SHr):	22.40 SHr
Price (THB, 1 SHr = 1.00 Baht):	22.40 Baht

Formula:

$$\text{SHr} = N_{\text{nodes}} \times [\alpha_{\text{cpu}} N_{\text{cores}} + \alpha_{\text{mem}} \text{Mem} + \alpha_{4500\text{gpu}} N_{4500\text{gpu}} + \alpha_{4000\text{gpu}} N_{4000\text{gpu}}] \times T_{\text{exec}}$$

where

T_{exec} : Execution time in hours,

N_{core} : Number of CPU cores allocated per node,

Mem : Memory size allocated per node, in GiB,

$N_{4500\text{gpu}}$: Number of RTX PRO 4500 GPUs allocated per node,

$N_{4000\text{gpu}}$: Number of RTX PRO 4000 GPUs allocated per node,

$\alpha_{\text{cpu}} (= 1.0)$: Weight per CPU processor core,

$\alpha_{\text{mem}} (= 0.1)$: Weight of memory size per GiB,

$\alpha_{4500\text{gpu}} (= 7.0)$: Weight per RTX PRO 4500 GPU accelerator,

$\alpha_{4000\text{gpu}} (= 5.0)$: Weight per RTX PRO 4000 GPU accelerator.

BistKA | KVIS Mini-HPC Job Credit (SHr) Usage Calculator
Created by Bundit Boonyarit, School of Information Science and Technology, VISTEC

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

BistKA Administration Team

BistKA Administration Team

Dr. Bundit Boonyarit



Current Position: Ph.D. Candidate (Information Science and Technology)

Current Affiliation: School of Information Science and Technology, VISTEC

Next Stop: Lecturer (Division of Preclinical Medicine), Chakri Naruebodindra Medical Institute (CNMI), Faculty of Medicine Ramathibodi Hospital, Mahidol University

Research Interests:

- Computational Biomolecular Design and Discovery
- Biomolecular Machine Learning
- Computer-aided Drug Discovery & Design
- Computational Chemistry & Biology
- Structural Bioinformatics

Email: bundit.b_s18@vistec.ac.th

Personal Website: <https://manbaritone.com/> 

Research Group Website: <https://baid.manbaritone.com/> 

Dr. Kanes Sumetpipat



Current Position: Mathematics Teacher

Current Affiliation: Department of Mathematics and Computer Science, KVIS

Research Interests:

- Applied Mathematics
- Machine Learning
- Optimization

Dr. Adis Khetubol



Current Position: Physics Teacher

Current Affiliation: Department of Physics and Astronomy, KVIS

Research Interests:

- Computational based General Physics and Applied Physics
- Machine Learning for Physics and Applied Science
- Molecular Dynamics and Quantum Chemistry Simulations of Molecular Materials

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)

How to Write Acknowledgement

How to Write Acknowledgement

For any student project, teacher project, or school class that uses the [BistKA](#) for advanced computing (including CPU, GPU, or high-memory workloads), if the outputs or results are presented in any form, such as a report, poster, or other materials, please include the acknowledgement text below.



The authors acknowledge KVIS Mini-HPC Cluster (BistKA) and AI & Mechatronics Center for their support of computing facilities.

© 2025 Kamnoetvidya Science Academy (KVIS). All rights reserved. | Powered by [Wiki.js](#)